

Q1. Why is the main method static?

The main method is **static** so the runtime system can call it to start a program without creating an object of the class first. [1]

Why main Needs to Be Static

- **No object needed:** A static method belongs to the class itself rather than to any specific instance.
- **Program entry point:** Because main is the very first code executed, no objects exist yet when the program starts.
- **Runtime access:** The Java Virtual Machine (JVM) or execution environment can directly locate and run the method using just the class name. If it were not static, the runtime would need to guess which constructor to use and instantiate an object before starting. [1, 2, 3]

Q2. Is Java pass-by-value?

Yes, Java is strictly pass-by-value. This means that whenever you pass an argument to a method, Java creates a **copy of the variable's value** and passes that copy into the method. Java never passes the actual reference or memory address of the variable itself (which would be pass-by-reference). [1, 2, 3, 4, 5]

The confusion usually arises because Java handles primitive types and object types differently, even though both use pass-by-value. [1, 2]

How it Works

Variable Type	What is the "Value"?	Behavior in a Method
Primitive Types (e.g., <i>int</i> , <i>boolean</i> , <i>char</i>)	The actual data itself (e.g., the number 5).	The method gets a copy of the data. Modifying it inside the method has no effect on the original variable outside.
Object Types (e.g., <i>String</i> , <i>ArrayList</i> , <i>Custom Objects</i>)	The memory address (pointer) that points to the object.	The method gets a copy of that memory address. Both the original variable and the method parameter now point to the same object .

Q3. How many ways can a Java object be created?

There are five main ways to create an object in **Java**. Each approach allocates memory for a new instance, though they differ in how they invoke constructors and handle initialization. [1, 2, 3]

Here is a breakdown of all five methods, along with code examples and behavioral differences: [1, 2]

1. Using the new Keyword (Most Common)

This is the standard and most intuitive approach. It directly allocates memory and explicitly invokes the targeted constructor (default or parameterized) to initialize the object. [1, 2, 3]

java

```
Employee emp = new Employee();
```

Use code with caution.

2. Using newInstance() of the Constructor Class (Reflection API)

Part of Java's reflection utility (`java.lang.reflect.Constructor`), this method allows you to fetch a specific constructor dynamically at runtime and create an instance. It is heavily used by frameworks like Spring and Hibernate. [1, 2, 3]

java

```
Constructor<Employee> constructor = Employee.class.getConstructor();
Employee emp = constructor.newInstance();
```

Use code with caution.

3. Using `newInstance()` of the `Class` Class (Reflection API)

An alternative reflection method involves fetching the class footprint via `Class.forName()` or `Employee.class` and instantiating it. [1]

- *Note:* `Class.newInstance()` is **deprecated** in modern Java versions because it can only invoke the no-argument constructor and propagates unhandled exceptions. It is recommended to use `Constructor.newInstance()` instead. [1, 2]

java

```
Employee emp = (Employee) Class.forName("com.example.Employee").newInstance();
// OR
Employee emp = Employee.class.newInstance();
```

Use code with caution.

4. Using the `clone()` Method

The `clone()` method copies an existing object's state into a brand new memory space (a shallow copy). Crucially, **no constructor is called** during this process. For this to work, the target class must implement the `Cloneable` marker interface and override the `clone()` method. [1, 2, 3]

java

```
Employee empCopy = (Employee) originalEmp.clone();
```

Use code with caution.

5. Using Deserialization

When you read a serialized object from a file or network stream using `ObjectInputStream.readObject()`, Java creates a brand new instance in memory. Like cloning, **no constructor is called** for the class being deserialized. The target class must implement the `Serializable` interface. [1, 2, 3]

java

```
ObjectInputStream in = new ObjectInputStream(new FileInputStream("data.obj"));
Employee emp = (Employee) in.readObject();
```

Q4. What is the superclass of all Java classes? Does a class extend `Object` plus another class?

The superclass of all Java classes is [java.lang.Object](#), and a class cannot directly extend `Object` plus another class at the same time. [1, 2]

The Superclass of All Classes

- **Root class:** `java.lang.Object` sits at the top of the Java inheritance tree.
- **Direct or indirect:** Every class in Java inherits from `Object` either directly or through an inheritance chain.
- **Implicit extension:** If you do not write an `extends` clause for your class, Java automatically makes it a direct subclass of `Object`. [1, 2, 3, 4]

Can a Class Extend `Object` and Another Class?

- **Single inheritance:** Java does not allow a class to extend more than one class directly.

- **No Object plus another:** If you write class Dog extends Animal, your Dog class directly extends Animal. It does *not* directly extend Object.
- **The chain:** Because Animal eventually extends Object (either directly or up the family tree), Dog still inherits from Object indirectly. [1, 2, 3, 4]

Q5. == vs equals() vs hashCode()

In Java, **==**, **equals()**, and **hashCode()** are all used to compare or identify objects, but they operate at fundamentally different levels.

Here is the direct breakdown: **==** is a binary operator that checks if two references point to the exact same memory address. **equals()** is a method designed to check for logical equality (matching content) between two objects. **hashCode()** is a method that converts an object's state into an integer value, primarily used to distribute objects across storage buckets in hash-based collections. [1, 2, 3, 4]

1. The == Operator

- **Primitives:** Compares actual values (e.g., 5 == 5 is true).
- **Objects:** Compares object **references** (memory locations). It only returns true if both variables point to the exact same object instance on the heap. [1]

java

```
String s1 = new String("hello");
String s2 = new String("hello");
System.out.println(s1 == s2); // false (different objects in memory)
```

Use code with caution.

2. The equals() Method

- This method is meant for **logical equality**. For example, two different String instances or two different Employee objects are "equal" if their internal fields contain identical data.
- Classes like String, Integer, and Double already override equals() to compare their wrapped values. [1, 2]

java

```
System.out.println(s1.equals(s2)); // true (their text content matches)
```

Use code with caution.

3. The hashCode() Method

- Instead of running a slow, field-by-field check using equals(), hash-based structures like [HashMap](#), HashSet, and Hashtable use hashCode() to find an object's approximate storage location ("bucket") instantly. [1, 2]
- It returns a 32-bit integer. If two objects have different hash codes, they are definitively not equal, allowing collections to skip calling equals() entirely. [1, 2]

Q6. final vs finally vs finalize()

In Java, final, finally, and finalize() are completely distinct concepts used for restriction, exception handling, and object cleanup. [1]

final (Keyword)

- **Purpose:** Used to restrict or apply immutability.
- **Variables:** Value cannot be reassigned once initialized.
- **Methods:** Cannot be overridden by subclasses.

- **Classes:** Cannot be inherited or extended. [\[1, 2\]](#)

finally (Block)

- **Purpose:** Used in exception handling with try-catch blocks to execute vital code.
- **Execution:** Guaranteed to run regardless of whether an exception is thrown or caught.
- **Usage:** Ideal for resource cleanup like closing database connections or streams. [\[1, 2\]](#)

finalize() (Method)

- **Purpose:** A method in the Object class called by the Garbage Collector before an object's memory is reclaimed.
- **Status: Deprecated** because it is unpredictable, slow, and unsafe; modern Java prefers try-with-resources or explicit cleanup. [\[1, 2\]](#)

Q7. transient vs volatile vs static keyword

In Java, **transient**, **volatile**, and **static** are modifiers used to control how variables behave in memory, across multiple threads, or during object saving (serialization). [\[1, 2, 3\]](#)

Detailed Breakdown

1. transient

The transient keyword is used in the context of **Object Serialization**. When you mark a variable as transient, you tell the Java Virtual Machine (JVM) to skip saving its value when the object is saved to a file, database, or sent over a network. [\[1, 2, 3\]](#)

- **Use Case:** Storing sensitive information like passwords, or temporary data like cached runtime values that don't need persistence.
- **Behavior:** When the object is deserialized (loaded back into memory), the transient variable is restored with its default type value (e.g., 0 for numbers, null for objects). [\[1, 2, 3\]](#)

2. volatile

The volatile keyword is used in **Multithreading**. By default, CPU threads cache copies of variables locally to speed up execution. If Thread A changes a variable, Thread B might not see that change right away because it is reading from its own cache. [\[1, 2, 3\]](#)

- **Use Case:** Managing flags or state indicators (like a private volatile boolean dataReady = false;) accessed by multiple threads.
- **Behavior:** Declaring a variable as volatile forces all threads to read and write its value directly from the **computer's main memory**. It ensures that any write operation "happens-before" subsequent reads. Note that it *does not* handle atomic modifications (like count++). [\[1, 2, 3, 4, 5\]](#)

3. static

The static keyword defines **Class-level ownership** rather than Instance-level ownership. Normally, each time you create an object, it gets its own unique copies of variables. A static variable is shared globally by every single instance of that class. [\[1, 2, 3, 4\]](#)

- **Use Case:** Global counters, constants (when combined with final), or configuration properties shared across all objects of that type.

- **Behavior:** Modifying a static variable from one object alters it for all other objects. However, in multithreaded environments, threads can still cache static variables locally unless they are also marked as volatile. [\[1, 2, 3\]](#)

Q8. What is Optional?

`java.util.Optional<T>` is a container object introduced in Java 8 that may or may not contain a non-null value. It was designed to provide a more elegant, compile-time solution for handling optional data instead of relying on null references, thereby reducing the frequency of `NullPointerException` (NPE) errors.

Why Use Optional?

Traditionally, programmers returned null to indicate a missing value. This forced calling methods to explicitly write null checks (`if (obj != null)`), which are easily forgotten and lead to runtime crashes. `Optional` forces the developer to think about the case where the value might be missing.

Q9. Iterate over a String without using predefined methods?

```
public class IterateWithoutMethods {
    public static void main(String[] args) throws Exception {
        String text = "Hello";

        // Access the private 'value' field inside the String class
        java.lang.reflect.Field field = String.class.getDeclaredField("value");
        field.setAccessible(true);

        // In modern Java, strings are backed by a byte[] (LATIN1 or UTF16)
        byte[] val = (byte[]) field.get(text);

        // Iterate through the byte array without calling charAt() or toCharArray()
        for (int i = 0; i < val.length; i++) {
            char c = (char) (val[i] & 0xFF);
            System.out.println(c);
        }
    }
}
```

Q10. Overloading vs overriding.

Overloading means writing multiple methods with the same name but different parameters in the same class, while overriding means rewriting a parent class method inside a child class with the exact same signature.

Key Differences

- **Scope:** Overloading happens within a single class. Overriding happens between a parent class and a child class through inheritance.
- **Parameters:** Overloaded methods must have different parameters (number, type, or order). Overridden methods must have the exact same parameters.
- **Polymorphism type:** Overloading is compile-time (static) polymorphism. Overriding is runtime (dynamic) polymorphism.

- **Return type:** Overloading can have any return type. Overriding requires the same or a covariant return type. [1, 2]

Q11. What happens when a static method is “overridden”? Method hiding

When a static method is "overridden" in a subclass with the same signature, it is not actually overridden; instead, method hiding occurs. [1, 2]

The Core Difference

The primary difference lies in **how the method call is resolved**:

- **Method Overriding (Instance Methods):** The version of the method that runs is decided at **runtime** based on the actual **object type**.
- **Method Hiding (Static Methods):** The version of the method that runs is decided at **compile-time** based on the **reference type** of the variable.

Q12. Overriding rules with exceptions, static and final methods

Static and final methods cannot be overridden in Java; instead, final methods are locked against modification, while static methods with the same signature perform "method hiding" rather than runtime polymorphism.

Final Methods

A method marked with the final keyword in a parent class cannot be overridden by any subclass.

- **Purpose:** Use final when you want to lock the implementation of a critical method and prevent subclasses from changing its behavior.
- **Restriction:** If a subclass tries to define a method with the exact same signature as a parent's final method, the compiler throws an error.

Static Methods and Method Hiding

Static methods belong to the class itself rather than an individual object instance.

- **No Overriding:** You cannot override a static method.
- **Method Hiding:** If a subclass defines a static method with the exact same signature as the parent class, it **hides** the parent's version instead of overriding it.
- **Resolution:** Execution of a hidden static method is decided at compile time based on the reference type (or class name), not at runtime based on the object type.

Exception Rules in Overriding

When you override a valid instance method, you must follow strict rules regarding exceptions in the throws clause:

- **Checked Exceptions:** An overriding method cannot throw a broader or new checked exception than the parent method. It can throw the same exception, a narrower (subclass) checked exception, or no checked exceptions at all.
- **Unchecked Exceptions:** An overriding method can freely throw any unchecked (runtime) exceptions regardless of what the parent method declares.

Q13. Covariant return types

A **covariant return type** allows an overriding method in a child class to return a more specific (derived) subtype than the return type declared in the parent class's overridden method.

How It Works

- **Parent class:** Defines a method with a general return type (such as an object or base class).
- **Child class:** Overrides that method and changes the return type to a subclass or derived type of the original return type.
- **Rules:** It only applies to reference types (or pointers/references in languages like C++), not to primitive data types. The method name and parameters (the signature) must stay exactly the same
- **No manual casting:** Callers who use the specific child class do not need to write explicit type casts to access methods unique to the child type.
- **Cleaner code:** It improves readability and maintenance across class hierarchies.

Q14. Abstract class vs interface

An abstract class allows code reuse with shared state and methods, while an interface defines a strict capability contract that a class must follow.

- **Inheritance:** A class can extend only one abstract class, but it can implement multiple interfaces.
- **State/Fields:** Abstract classes can have instance variables and constructors. Traditional interfaces cannot hold instance state and only contain constants.
- **Methods:** Abstract classes can have both abstract (unimplemented) and concrete (fully defined) methods. Interfaces traditionally only declare method signatures, though modern languages support default implementations.
- **Use an abstract class when:** You share code among closely related classes or need to maintain common state (fields or constructors) across a hierarchy.
- **Use an interface when:** You want to define common behavior or capabilities for unrelated classes or need multiple inheritance of type.

Q15. Default and static methods in interfaces

Core Comparison

Feature	Default Methods	Static Methods
Keyword	default	static
Invocation	Called via an instance of the implementing class.	Called via the interface name directly.
Overriding	Can be overridden by the implementing class.	Cannot be overridden by the implementing class.
Inheritance	Inherited automatically by implementing classes.	Not inherited by implementing classes or sub-interfaces.
Primary Purpose	To evolve interfaces without breaking	To provide utility or helper methods

existing implementations.

related to the interface.

1. Default Methods

- Default methods provide a fallback implementation. If a class implements an interface, it automatically inherits this method and its behavior unless it explicitly overrides it. [\[1, 2, 3, 4\]](#)

```
interface Vehicle {
```

```
void start(); // Abstract method
```

```
// Default method providing a standard implementation
```

```
default void blowHorn() {
```

```
    System.out.println("Blowing horn...");
}
```

```
class Car implements Vehicle {
```

```
    public void start() {
        System.out.println("Car engine started.");
    }
    // blowHorn() is automatically available without overriding
}
```

2. Static Methods

Static methods in interfaces behave identically to static methods in classes. They belong entirely to the interface itself and cannot be invoked through a class instance. This eliminates the need to create separate utility helper classes (like Collections or Math). [\[1, 2, 3, 4\]](#)

```
interface MathUtils {
```

```
    // Static method acting as a utility helper
```

```
static int add(int a, int b) {
```

```
    return a + b;
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
        // Invoked directly using the interface name
        int sum = MathUtils.add(5, 10);
        System.out.println("Sum: " + sum); // Output: 10
    }
}
```

Q16. Diamond problem with two interface default methods having the same signature

When a class implements two independent interfaces that define a **default method with the exact same signature**, the Java compiler cannot automatically resolve the ambiguity. This creates a compilation error known as the **interface diamond problem**. [\[1, 2\]](#)

The Fix: The implementing class **must override** the conflicting method to resolve the ambiguity. [1, 2]

```
interface InterfaceA {  
  
    default void log() { System.out.println("Log A"); }  
}  
  
interface InterfaceB {  
  
    default void log() { System.out.println("Log B"); }  
}  
  
class MyClass implements InterfaceA, InterfaceB {  
  
    // Compiler error unless we override and resolve the conflict:  
    @Override  
    public void log() {  
  
        InterfaceA.super.log(); // Explicitly chooses InterfaceA's implementation  
    }  
}
```

The 3 Core Resolution Rules in Java

When managing default methods, Java evaluates conflicts using these three strict precedence rules: [1, 2]

1. **Classes Always Win:** A method implementation declared in a class or a superclass always takes absolute priority over any interface default method. [1, 2]
2. **Sub-interfaces Win:** If one interface extends the other (e.g., interface B extends A), the more specific child interface's default method is automatically selected. [1, 2]
3. **Explicit Override Required:** If the interfaces are entirely independent (like InterfaceA and InterfaceB above) and no class-level implementation exists, the compiler throws an error until you provide an explicit override. [1, 2]

Q17. Composition vs inheritance

Composition builds complex types by combining independent objects, whereas inheritance creates new classes by deriving them from existing parent classes. [1, 2]

Key Differences

- **Relationship Type:** Inheritance models an "**is-a**" relationship (e.g., a Dog is an Animal), while composition models a "**has-a**" or "**uses-a**" relationship (e.g., a Car has an Engine). [1, 2]
- **Coupling:** Inheritance is **tightly coupled** because changes to a parent class ripple down and can break child classes. Composition is **loosely coupled** because components interact through clean interfaces or parameters. [1, 2, 3]
- **Reusability and Flexibility:** Inheritance duplicates or forces entire hierarchies of code, which can lead to rigid "god classes." Composition pieces together small, single-responsibility behaviors that can be swapped dynamically. [1, 2, 3]

Why Favor Composition Over Inheritance?

- Most experienced developers follow the principle of "**composition over inheritance**". Inheritance introduces the fragile base class problem, where updating a superclass causes unintended side effects across the entire tree of subclasses. Composition avoids this by letting you assemble features using modular building blocks. [1, 2]

Q18. Association vs Aggregation vs Composition

Association, aggregation, and composition are three levels of structural relationships in object-oriented programming and [UML Association vs Aggregation vs Composition](#), ordered from general to strictly lifecycle-dependent. [1, 2]

Association

Association is a general, broad connection between two separate classes. [1]

- **Definition:** It shows that two objects know about each other and communicate.
- **Ownership:** Neither object owns the other.
- **Lifecycle:** Both objects live and die independently.
- **Example:** A **Doctor** and a **Patient**. A doctor sees many patients, and a patient can see many doctors. If you delete a doctor object, the patient object still exists. [1]

Aggregation

Aggregation is a special, stronger type of association. It represents a "has-a" relationship. [1, 2]

- **Definition:** One object (the whole) contains or holds a reference to another object (the part).
- **Ownership:** The whole owns the part, but the part can also belong to other wholes or be shared.
- **Lifecycle:** The child part can exist independently of the parent whole. If you destroy the parent, the child survives.
- **Example:** A **Library** and a **Book**. A library has books. If the library closes down and is deleted, the books are not destroyed and can move to a new library. [1, 2]

Composition

Composition is a strict and much stronger form of aggregation. [1, 2]

- **Definition:** It is a whole-part relationship where the part belongs exclusively to a single whole.
- **Ownership:** The parent object has exclusive ownership of the child object.
- **Lifecycle:** The child cannot exist without the parent. If the parent object is destroyed, the child object is destroyed with it.
- **Example:** A **House** and a **Room**. A house is composed of rooms. If you demolish the house, the rooms inside it cease to exist

Q19. What is a Singleton class?

A **Singleton class** is a software design pattern that guarantees a class has only one instance and provides a global point of access to that instance. It is commonly used for managing shared resources, such as database connections, logging services, or configuration settings.

Key Characteristics

- **Single Instance:** The class restricts itself from being instantiated more than once throughout the application's lifecycle.
- **Global Access:** The single instance is easily accessible across different parts of the application.
- **Private Constructor:** The class hides its constructor to prevent other classes from using the new keyword to create new objects.
- **Static Instance & Method:** The class holds a static variable of its own type and provides a public static method (usually named `getInstance()`) that returns the single instance.

```
public class Singleton {
    // 1. Create a private static instance of the class
    private static Singleton instance;

    // 2. Make the constructor private so it cannot be instantiated outside
    private Singleton() {}

    // 3. Provide a public static method to get the instance
    public static Singleton getInstance() {
        if (instance == null) {

            instance = new Singleton(); // Lazy initialization
        }
        return instance;
    }
}
```

Q20. What is Singleton in a multithreaded environment: double-checked locking?

Double-checked locking is a design pattern used to make the [Singleton Design Pattern](#) thread-safe and fast by reducing the overhead of synchronization. [1, 2]

How Double-Checked Locking Works

Instead of locking the entire method every time it is called—which hurts performance—the code checks if the instance exists *before* acquiring a lock. [1, 2]

- **First Check (No Locking):** Threads check if the instance is null without any synchronization. If the object already exists, threads skip the lock and return the instance instantly, maximizing speed. [1, 2, 3]
- **The Lock:** If the first check finds the instance is null, threads enter a synchronized block. [1, 2]
- **Second Check (With Locking):** Inside the locked block, the code checks if the instance is null a second time. This ensures that if two threads passed the first check at the same time, only the first thread creates the object while the second thread sees it is now initialized. [1, 2]

```
public class ThreadSafeSingleton {
    // The volatile keyword is critical here!
    private static volatile ThreadSafeSingleton instance;

    // Private constructor prevents external instantiation
    private ThreadSafeSingleton() {}

    public static ThreadSafeSingleton getInstance() {
        // First Check: Avoids synchronization overhead once initialized
        if (instance == null) {
            synchronized (ThreadSafeSingleton.class) {
                // Second Check: Prevents multiple instances if threads entered concurrently
            }
        }
    }
}
```

```

        if (instance == null) {

            instance = new ThreadSafeSingleton();

        }
    }
}
return instance;
}
}

```

Q21. If a Singleton implements Serializable, what is the impact?

If a **Singleton** class implements **Serializable** in Java, its singleton property is broken because **deserialization creates a new instance** of the class every time an object is read from a stream.

By default, the `ObjectInputStream.readObject()` method bypasses the private constructor and instantiates a brand new object in memory, violating the core rule of the Singleton pattern (which guarantees only one instance exists).

Q22. How do you resolve the Singleton serialization issue?

To maintain the Singleton guarantee during serialization, you must implement the **readResolve()** method or use an **Enum**.

1. The readResolve() Method

You can define a `readResolve()` method within your Singleton class. The JVM automatically invokes this method during deserialization, allowing you to replace the newly created object with your existing instance.

java

```

import java.io.Serializable;

public class SerializableSingleton implements Serializable {
    private static final long serialVersionUID = 1L;

    private SerializableSingleton() {}

    private static class Holder {
        private static final SerializableSingleton INSTANCE = new SerializableSingleton();
    }

    public static SerializableSingleton getInstance() {
        return Holder.INSTANCE;
    }

    // Enforces the singleton guarantee during deserialization
    protected Object readResolve() {
        return getInstance();
    }
}

```

Use code with caution.

2. The Enum Solution (Best Practice)

The most robust way to implement a serializable singleton is to use an **enum**. The Java language inherently guarantees that enums are properly serialized and deserialized without creating duplicate instances, completely preventing reflection or serialization attacks.

```
java
public enum EnumSingleton {
    INSTANCE;

    // Your logic and fields go here
}
```

Q23. Why are Strings immutable?

In Java and many other programming languages, Strings are immutable (cannot be changed after they are created) primarily for security, optimization, and thread safety.

Here is a breakdown of why this design choice is critical:

1. The String Pool (Memory Optimization)

- **Space Saving:** Java uses a special memory region called the String Constant Pool. When you create a string literal, the JVM checks the pool first. If the string already exists, it reuses the reference instead of creating a new object.
- **The Risk of Mutability:** If Strings were mutable, changing the value of a string through one reference would silently change it for all other references pointing to the same memory location, corrupting data across the application.

2. Security

- **Network & File Operations:** Strings are heavily used to store sensitive data like file paths, database URLs, usernames, and passwords.
- **Preventing Alteration:** If Strings were mutable, an attacker could pass a valid file path helper or network address into a security check, and then alter the underlying string content immediately afterward (a Time-of-Check to Time-of-Use vulnerability).

3. Thread Safety

- **No Synchronization Needed:** Because immutable objects cannot be modified, they can be shared freely across multiple threads without the risk of data corruption or race conditions. This simplifies concurrent programming.

4. Caching HashCodes

- **Performance Boost:** Strings are frequently used as keys in hash-based collections like HashMap or HashSet.
- **Guaranteed Consistency:** Because the content of a String never changes, its hashCode() only needs to be calculated once and can be cached. If Strings were mutable, changing the string content would alter its hashCode, meaning you would lose the ability to find your object inside a map.

Q24. How do you design an immutable class?

To design an **immutable class** (a class whose state cannot be changed after it is created), you must adhere to core architectural rules that lock its state and prevent external leakage. While minor implementation details vary by language, the fundamental requirements remain highly consistent. [1]

Core Rules for Designing an Immutable Class

1. Prevent Inheritance (final class)

Declare the class as final (or your language's equivalent) so it cannot be subclassed. This prevents a child class from overriding methods, adding mutable state, or altering the class's original behavior.

0 *Alternative:* Make all constructors private and use a public static factory method to generate instances. [1, 2, 3, 4, 5]

2. Restrict Field Access (private and final / readonly)

Make all fields **private** so they cannot be accessed directly from outside the class. Make them **final** (Java) or **readonly** (C#) so they can only be assigned once—either during declaration or within the constructor. [1, 2, 3]

3. Do Not Provide Setters

Omit any methods that modify fields or objects referred to by fields (e.g., no setX() methods). Provide only **getter methods** to read data. [1, 2, 3]

4. Initialize All Fields via a Parameterized Constructor

Set the complete state of the object exactly once during initialization. [1]

5. Perform Defensive Copying on Mutable Fields (Critical)

If your class contains references to mutable objects (like arrays, lists, or custom objects), basic wrappers are not enough. You must isolate them:

- o **In the constructor:** Do not assign external object arguments directly. Instead, create a **deep copy** (defensive copy) of the passed object and assign the copy to your internal field.

- o **In the getters:** Never return the original reference to an internal mutable field. Instead, clone or create a **defensive copy** of the object and return that copy. This ensures callers cannot bypass the class constraints and mutate your internal state from the outside. [1, 2, 3, 4, 5]

Q25. Create an immutable Employee class containing Address as an instance variable

To create a truly **immutable class in Java**, you must ensure that its state cannot be modified after construction. When a class contains a mutable object like a custom Address class as an instance variable, you must perform **deep copying (defensive copying)** in both the constructor and the getter methods to prevent external modification.

1. The Mutable Address Class

```
public class Address {  
    private String city;  
    private String state;
```

```
// Constructor
```

```
public Address(String city, String state) {  
    this.city = city;  
    this.state = state;  
}
```

```
// Copy Constructor (used for defensive copying)
```

```
public Address(Address address) {  
    this.city = address.city;  
    this.state = address.state;  
}
```

```
// Getters and Setters
```

```
public String getCity() { return city; }  
public void setCity(String city) { this.city = city; }
```

```
public String getState() { return state; }  
public void setState(String state) { this.state = state; }
```

```
@Override
```

```
public String toString() {  
    return city + ", " + state;  
}
```

```
}
```

2. The Immutable Employee Class

To make this class immutable, we follow these rules:

- Declare the class as **final** so it cannot be extended.
- Make all fields **private final**.
- Do not provide **setter** methods.
- Perform **defensive copying** in the constructor and the getter for Address.

```
public final class Employee {
    private final int id;
    private final String name;
    private final Address address; // Mutable instance variable

    // Constructor
    public Employee(int id, String name, Address address) {
        this.id = id;
        this.name = name;
        // Defensive copy: creates a new Address object so changes to the original don't affect Employee
        this.address = new Address(address);
    }

    // Getters
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Address getAddress() {
        // Defensive copy: prevents the caller from modifying the internal Address object
        return new Address(this.address);
    }

    @Override
    public String toString() {
        return "Employee{id=" + id + ", name=" + name + ", address=" + address + "}";
    }
}
```

Q26. What is a Marker interface?

A **marker interface** is an interface that contains no methods, constants, or fields. It is also known as a **tagging interface** because it is used simply to "tag" or mark a class, signaling to the runtime environment, compiler, or a framework that the class possesses a specific property or behavior.

When a class implements a marker interface, it is telling the JVM (Java Virtual Machine) or a specific library: *"I am allowed to do this"* or *"Treat me specially."* The code then uses the instanceof operator to check for the presence of the interface before performing an operation.

Common Examples in Java

- java.io.Serializable: Tells the JVM that an object's state can be converted into a byte stream (serialized).
- java.lang.Cloneable: Signals that the Object.clone() method is allowed to make a field-for-field copy of instances.
- java.util.RandomAccess: Indicates that a collection supports fast, constant-time random access (like ArrayList vs LinkedList).

Modern Alternative

In modern Java (Java 5 and later), **Annotations** (like `@Override` or custom annotations) have largely replaced marker interfaces. Annotations are generally preferred because they are more flexible, can take parameters, and do not require changing the type hierarchy of the class.

Q27. String pool and String immutability

In Java, **String immutability** and the **String Pool** are deeply intertwined concepts designed to optimize memory, enhance security, and improve performance.

Here is a comprehensive breakdown of how they work, how they rely on each other, and why this design matters.

1. What is String Immutability?

String immutability means that once a String object is created in memory, its value **cannot be changed**. [1, 2]
When you perform an operation that appears to modify a string (such as `concat()`, `replace()`, or `toLowerCase()`), the Java Virtual Machine (JVM) does not alter the original object. Instead, it allocates a **completely new String object** in memory with the updated content, leaving the original untouched. [1, 2]

How Immutability is Enforced Internally:

- **final Class:** The String class is declared as public final class String, meaning it cannot be extended. This prevents malicious subclasses from overriding its behaviors.
- **final Character Array:** Internally, the characters are stored in a private byte or character array (e.g., private final byte[] value;). The final keyword ensures the reference to this array cannot be reassigned after construction.
- **No Setters:** The class does not expose any public methods that allow modifying the internal array. [1, 2]

2. What is the String Pool?

The **String Pool** (or String Literal Pool) is a special storage area inside the **Java Heap Memory**. It acts as a cache for string literals to drastically optimize memory usage. [1, 2]

When you create a string using a literal, the JVM checks the String Pool first: [1]

1. If the string value **already exists** in the pool, the JVM returns a reference to the existing object instead of creating a duplicate.
2. If it **does not exist**, a new string object is created and placed inside the pool. [1, 2]

```
String s1 = "Hello"; // Created in the pool
String s2 = "Hello"; // Reuses s1's instance from the pool
System.out.println(s1 == s2); // true (Points to the exact same memory address)
```

The new Keyword Exception:

If you bypass literals and use the new keyword, you explicitly force the JVM to create a fresh, unique object outside the pool in the standard heap space, even if that text already exists in the pool. [1, 2]

```
String s3 = new String("Hello"); // Created outside the pool on the heap
System.out.println(s1 == s3); // false (Different memory addresses)
```

3. The Relationship: Why the Pool Requires Immutability

The String Pool could not safely exist without String Immutability. [1]

Because multiple reference variables point to the exact same object in the pool, mutability would cause severe data corruption. [1]

- **The Risk:** Imagine if s1 and s2 both point to "Hello" in the pool. If strings were mutable, and a developer changed s1 to "Mello", the value of s2 would automatically change to "Mello" as well.

- **The Solution:** Immutability guarantees that one application component or thread cannot accidentally ruin data being shared by another

4. Key Advantages of this Design

Feature	How Immutability & Pooling Help
Memory Optimization	Prevents the heap from filling up with millions of duplicate strings. Multiple references share a single pool instance.
Thread Safety	Since string objects cannot change, they are inherently thread-safe . Multiple threads can share them simultaneously without requiring complex synchronization code.
Security	Strings are heavily used for sensitive parameters like database URLs, passwords, and network connections. If strings were mutable, a malicious exploit could modify a connection string <i>after</i> validation checks passed, changing the application's behavior.
HashCode Caching	Strings are the most common keys used in HashMap and HashSet. Because a string never changes, its hashCode() is calculated once during creation and cached. This makes hash-table lookups incredibly fast.

Q28. String vs StringBuilder vs StringBuffer

In Java, [String](#) represents an immutable sequence of characters, while [StringBuilder](#) and [StringBuffer](#) are mutable, with [StringBuffer](#) offering thread safety through synchronization. [1, 2]

Key Differences

- **Mutability:** String objects cannot be changed after creation; any modification creates a new object. Both [StringBuilder](#) and [StringBuffer](#) allow in-place modifications without creating new objects. [1, 2]
 - **Thread Safety:** String is inherently thread-safe due to immutability. [StringBuffer](#) is synchronized and thread-safe. [StringBuilder](#) is not synchronized and is not thread-safe. [1, 2, 3]
 - **Performance:** String is slow for frequent modifications. [StringBuilder](#) is the fastest for single-threaded operations. [StringBuffer](#) is slower than [StringBuilder](#) because of synchronization overhead. [1, 2]
 - **String:** Use when the text is constant or unchanging.
 - **StringBuilder:** Use for heavy string manipulation and concatenation in single-threaded environments.
 - **StringBuffer:** Use when multiple threads share and modify the same string sequence concurrently. [1, 2, 3, 4]
- A [HashMap](#) in Java internally stores data as key-value pairs inside an array of buckets, using hashing and linked lists or trees to achieve average $O(1)$ time complexity for insertions and lookups. [1, 2, 3]

⌘ **Node Array:** The core structure is an array of Node objects, often called the bucket array or table.

⌘ **Node Structure:** Each node contains four fields: hash, key, value, and next (a pointer to the next node).

⌘ **Capacity and Load Factor:** The default initial capacity is 16, and the default load factor is 0.75. Resizing occurs when capacity exceeds 75%. [1, 2, 3]

How put() Works

1. **Calculate Hash:** The hashCode() method is called on the key, followed by a bit-spreading function to optimize the distribution. [1, 2]
2. **Find Bucket Index:** A formula using bitwise operations $((n - 1) \& \text{hash})$ maps the hash code to a specific array index (bucket). [1]

3. Handle Collisions:

- o If the bucket is empty, the new node is placed there.
 - o If the bucket is occupied (a collision), the keys are compared using equals(). If the key already exists, the old value is overwritten. If it is a new key, it is appended to a linked list at that bucket. [1, 2]
4. **Treeification (Java 8+):** If a bucket's linked list exceeds 8 nodes and total capacity is at least 64, the linked list converts into a balanced Red-Black tree to improve worst-case search times from $O(n)$ to $O(\log n)$. [1, 2]

How get() Works

1. **Compute Hash and Index:** The map calculates the key's hash and determines the bucket index using the same logic as `put()`.
2. **Traverse and Compare:** It inspects the target bucket, comparing hashes and checking keys with `equals()` to find the exact match and return its value. [1]

Q30. **HashMap** vs **synchronizedMap** vs **ConcurrentHashMap**

HashMap is not thread-safe and offers fast single-threaded performance, **synchronizedMap** locks the entire map for every operation, and **ConcurrentHashMap** uses fine-grained bin-level locking for high-concurrency performance. [1, 2, 3, 4, 5]

Key Differences

- **Thread Safety:**
 - o **HashMap:** Not thread-safe; unsafe for multi-threaded access.
 - o **synchronizedMap:** Thread-safe via a `synchronized` wrapper on a `HashMap`.
- o **ConcurrentHashMap:** Thread-safe natively designed for concurrent execution. [1, 2, 3, 4, 5]
- **Locking Strategy:**
 - o **HashMap:** No locks.
 - o **synchronizedMap:** Object-level locking (locks the entire map instance for every read/write).
- o **ConcurrentHashMap:** Granular bucket/node-level locking and non-blocking reads. [1, 2, 3, 4, 5]
- **Null Support:**
 - o **HashMap:** Allows one null key and multiple null values.
 - o **synchronizedMap:** Allows one null key and multiple null values.
- o **ConcurrentHashMap:** Does not permit null keys or values (throws `NullPointerException`). [1, 2]

Iteration Behavior:

- **HashMap & synchronizedMap:** Fail-fast iterators that throw `ConcurrentModificationException` if modified structurally during iteration.
- **ConcurrentHashMap:** Weakly consistent (fail-safe) iterators that do not throw `ConcurrentModificationException`. [1, 2, 3]

Q31. How can you put all elements into the same **HashMap** bucket?

To put all elements into the same **HashMap** bucket, you must make all their keys produce the same hash code.

How to Force a Hash Collision

- **Override `hashCode()`:** Create a custom key object and make its `hashCode()` method return a constant number, such as 0 or 1.
- **Understand the Bucket Index:** Java calculates the bucket index using the formula $(n - 1) \& \text{hash}$, where n is the array capacity. If every key returns the same hash code, every key maps to the exact same index.
- **Observe Worst-Case Performance:** When all elements fall into one bucket, the **HashMap** degrades from $O(1)$ time complexity to a linked list or tree traversal with $O(n)$ time complexity.

```
public class SameBucketKey {  
    private String id;
```

```
    public SameBucketKey(String id) {  
        this.id = id;  
    }
```

```
    @Override  
    public int hashCode() {  
        return 42; // All instances share this same hash code  
    }
```

```

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    SameBucketKey that = (SameBucketKey) obj;
    return id.equals(that.id);
}
}

```

Q32. hashCode() and equals() contract

⚠ **Equality Rule:** If `a.equals(b)` is true, then `a.hashCode()` must equal `b.hashCode()`.

⚠ **No Reverse Rule:** If `a.hashCode()` equals `b.hashCode()`, the objects do **not** have to be equal. (This is called a hash collision).

⚠ **Consistency Rule:** As long as an object's data does not change, calling `hashCode()` multiple times must return the same integer value.

🔧 **Override Rule:** Whenever you override `equals()`, you **must** also override `hashCode()`. [\[1, 2, 3\]](#)

Q33. How many default buckets and what load factor does HashMap have?

A Java HashMap has a default initial capacity of 16 buckets and a default load factor of 0.75. [\[1\]](#)

Key Details

- **Default Buckets (Initial Capacity):** 16 memory locations or bins are created when you initialize an empty map with `new HashMap<>()`.
- **Default Load Factor:** 0.75 (or 75%). This means the map resizes and doubles its bucket capacity once the number of stored entries exceeds $(16 \times 0.75 = 12)$ items.
- **Trade-off:** A load factor of 0.75 provides a good balance between time cost (fewer collisions during lookups) and space cost (memory usage).

Q34. Why do we need multiple buckets in HashMap?

We need multiple buckets in a HashMap to prevent different keys from mapping to the exact same storage location, which keeps data lookup and insertion fast. [\[1, 2\]](#)

Why Multiple Buckets Matter

- **Handling Collisions:** A hash function turns a key into a number (hash code). Because a map can hold many keys but array sizes are limited, different keys can produce the same index. This situation is called a **collision**. Multiple buckets give different collisions separate spaces to live instead of forcing everything into one spot. [\[1, 2, 3\]](#)
- **Maintaining Speed (O(1) Time):** If a HashMap had only one bucket, it would just be a single linked list or array where you have to check every single item to find a match. Searching through a single list takes $O(n)$ time (linear time, meaning it gets slower the more items you add). Spreading items across multiple buckets keeps the average search time at $O(1)$ (constant time, meaning it stays fast no matter how big the map gets). [\[1, 2, 3\]](#)
- **Managing Overflow via Chaining:** When a collision does happen and two keys land in the same bucket, the bucket holds them together in a small chain (using a linked list or a tree). Having multiple buckets ensures these chains stay short and efficient. [\[1, 2, 3\]](#)

Q35. How does HashSet work internally?

In Java, a HashSet works internally by wrapping a HashMap. Even though a HashSet appears to store individual values, it delegates all its core storage, uniqueness, and lookup logic directly to an internal HashMap instance. [\[1, 2, 3, 4\]](#)

Whenever you instantiate a HashSet, it initializes an internal HashMap. Because a map requires **key-value pairs**, the HashSet adapts its single elements to fit this structure: [\[1, 2\]](#)

- **The Key:** The element you want to store in the HashSet acts as the **Key** in the HashMap.

- **The Value:** A private, static, final dummy object named **PRESENT** is used as a constant value for every single entry. [1, 2, 3]

2. How add() Works and Guarantees Uniqueness

- When you call add(E e), the HashSet invokes the put(K key, V value) method on the internal map. [1, 2]
The underlying HashMap ensures uniqueness based on how put() handles keys: [1, 2]
- 1. **Hashing & Bucket Location:** The HashMap calculates the element's hash code via hashCode() to find its array bucket. [1, 2]
- 2. **Checking Duplicates:** If other elements exist in that bucket, HashMap traverses them using equals() to check for an exact match. [1]
- 3. **The Return Value Logic:**
 - o If the key is **new**, map.put() inserts it and returns **null**. The add() method checks this, evaluates null == null as true, and successfully reports the element was added.
 - o If the key **already exists**, map.put() overwrites the old dummy value with the new PRESENT object and returns the *previous* value (which is PRESENT). Because PRESENT == null is false, the add() method returns false and ignores the duplicate. [1, 2, 3]

3. How Other Core Methods Work

Because HashSet is just a wrapper, all other basic operations are pass-through calls to the HashMap: [1]

Method	Internal Implementation	Behavior
contains(Object o)	map.containsKey(o)	Leverages O(1) hash lookups to see if the element exists as a key.
remove(Object o)	map.remove(o) == PRESENT	Removes the key from the map. Returns true if the map returned PRESENT.
clear()	map.clear()	Empties the internal map array buckets.
iterator()	map.keySet().iterator()	Iterates over the keys of the map. It is fail-fast , throwing a ConcurrentModificationException if modified mid-loop.

4. Handling Performance and Collisions

Since it relies entirely on HashMap, it inherits all of its structural mechanics: [1]

- **Time Complexity:** Average time complexity for add, remove, and contains is **O(1)**. [1, 2]
- **Collision Resolution:** When different elements share the same bucket, they form a linked list. In modern Java (Java 8+), if a bucket exceeds **8 elements** and the total capacity is at least 64, the list converts into a **Red-Black Tree** to keep performance at $O(\log n)$. [1, 2, 3]
- **Resizing:** When the number of entries surpasses 75% (default load factor of 0.75) of the total bucket capacity, the bucket array automatically **doubles in size**. [1, 2]

Q36. Fail-Fast vs “Fail-Safe” iterators

Fail-fast iterators throw a ConcurrentModificationException if a collection is modified structurally during iteration, while fail-safe iterators operate on a snapshot or clone of the collection and never throw this exception. [1, 2, 3]

Key Differences

- **Behavior on Modification:**
 - o **Fail-Fast:** Immediately stops and throws ConcurrentModificationException if an element is added, removed, or modified directly (outside the iterator's own remove method).
- o **Fail-Safe:** Does not throw an exception; it continues iterating over a separate copy or snapshot of the data created when the iterator was made. [1, 2, 3, 4]
- **Underlying Design:**
 - o **Fail-Fast:** Uses a modification count (modCount) variable and checks it during traversal. It operates directly on the original collection.
- o **Fail-Safe:** Works on a cloned or concurrent copy of the original data structure. [1, 2, 3]
- **Memory and Performance:**

- o **Fail-Fast:** Uses less memory and offers better performance because it does not clone data.
- o **Fail-Safe:** Uses more memory and can have higher overhead due to cloning or synchronization. [1]

- **Examples in Java:**

- o **Fail-Fast:** [ArrayList](#), [HashMap](#), and most standard collections in [java.util](#).
- o **Fail-Safe:** [CopyOnWriteArrayList](#), [ConcurrentHashMap](#), and other classes in [java.util.concurrent](#). [1, 2]

Q37. What happens if I add a new element to a Map while iterating it?

If you attempt to add a new element directly to a standard Map while iterating over it, Java will throw a **ConcurrentModificationException**. [1, 2]

This happens because standard Map implementations (like [HashMap](#), [TreeMap](#), or [LinkedHashMap](#)) use **fail-fast iterators**. These iterators monitor an internal structural modification count (`modCount`). If they detect that the map's structural composition changed outside of the iterator's control during progress, they immediately abort to prevent unpredictable, erratic behavior. [1, 2, 3]

1. Use a Temporary Map (Best for Standard HashMaps)

Iterate through your existing map normally, but store any new elements in a separate temporary map. Merge them together once the iteration concludes. [1, 2]

2. Use ConcurrentHashMap (Best for Multi-threaded or Real-time Additions)

If your architecture permits, switch to a [ConcurrentHashMap](#). Its iterators are **fail-safe**, meaning they do not throw a **ConcurrentModificationException** when structural changes occur. [1, 2]

3. Iterate over a Copied Collection

You can iterate over a shallow copy of the map's keys or entries. Because you are traversing the clone, modifying the original map will not conflict with the active loop. [1]

Q38. ConcurrentHashMap bucket-level locking in java ?

In Java 8 and later versions, [ConcurrentHashMap](#) achieves high concurrency through a mechanism called bucket-level locking (or bin-level locking) combined with lock-free techniques. [1, 2]

Instead of grouping buckets into a few distinct segments, the map uses a volatile array of Node elements (`table[]`). Thread safety for write operations (`put`, `remove`, `compute`) is handled on a per-bucket basis: [1, 2, 3]

1. Reads are Always Lock-Free

All read operations (`get`, `containsKey`, `size`) do not require any locks. Because the node table is declared volatile, threads always read the freshest data directly from main memory without blocking. [1, 2]

2. CAS for Empty Buckets (Lock-Free Inserts)

When a thread attempts to write to a bucket that is currently **empty (null)**, it does not acquire a lock. Instead, it uses a low-level, atomic **Compare-And-Swap (CAS)** instruction: [1, 2, 3]

- The thread checks if the bucket is still null.
- If it is, the thread atomically places the new node into the bucket.
- If another thread slips in first, the CAS operation fails, and the thread simply retries without ever blocking. [1, 2, 3]

3. Synchronizing Only on the Head Node (Bucket-Level Lock)

If a thread attempts to write to a bucket that **already contains elements** (a hash collision resulting in a linked list or Red-Black Tree): [1, 2]

- The thread uses a standard Java synchronized block, locking **only the first (head) node** of that specific bucket.
- Because the lock is isolated to a single head node, any other threads trying to read or write to **different buckets** can proceed concurrently without waiting. [1, 2, 3]

Q39. Array vs ArrayList

In Java, an [Array vs ArrayList](#) comes down to size flexibility, data types supported, and built-in features. An array has a fixed size set when created, while an [ArrayList](#) can grow or shrink dynamically. [1]

- * **Size:** Arrays are static and cannot change length after creation. [ArrayLists](#) are dynamic and resize automatically when needed.

⌘ **Data Types:** Arrays store both primitive types (like `int`, `char`) and objects. `ArrayLists` store only objects, requiring wrapper classes (like `Integer`) for primitives.

⌘ **Performance:** Arrays are faster and use less memory. `ArrayLists` have minor overhead from resizing and autoboxing.

⌘ **Methods:** Arrays have no built-in manipulation methods and use `.length`. `ArrayLists` offer many methods like `.add()`, `.remove()`, and `.size()`

Q40. If an array has fixed size, why use an array instead of `ArrayList`?

Even when your data has a fixed size, using a standard **array** instead of an **`ArrayList`** provides major advantages in performance, memory consumption, syntax clarity, and primitive support.

1. Direct Support for Primitive Data Types

- **Array:** Can store **primitives directly** (e.g., `int[]`, `char[]`, `double[]`). The data is packed contiguously in memory, making it incredibly fast to access and process. [1, 2, 3, 4]
- **`ArrayList`:** Can **only store object types** (`Integer`, `Character`, `Double`). If you pass a primitive, Java must automatically wrap it into an object via **autoboxing**. This adds a layer of indirection (pointers to objects on the heap) and significantly degrades performance during heavy math or iterations. [1, 2, 3, 4]

2. Lower Memory Overhead

- **Array:** A fixed-size array allocates exactly the memory required for its length, plus a minor object header.
- **`ArrayList`:** Every `ArrayList` is a wrapper class that includes an internal object reference array, integer trackers for the list size, and individual object overhead for every element stored (if using primitives). Furthermore, an `ArrayList` often over-allocates its internal capacity to prepare for growth, resulting in unused memory overhead. [1, 2, 3, 4, 5]
- **3. Faster Execution and Cache Locality**
- Because a primitive array stores its values sequentially in a single contiguous block of memory, it plays very nicely with the CPU cache. A CPU can load chunks of the array into its high-speed cache all at once. An `ArrayList` of objects, by contrast, stores an array of *pointers* to objects scattered across the heap, forcing the CPU to repeatedly look up random memory locations (pointer chasing). [1, 2, 3, 4]

4. Multidimensional Capabilities

- **Array:** Native support for multidimensional arrays (e.g., `int[][] matrix = new int[3][3]`), which is clean and efficient for representing coordinates, grids, or matrices.
- **`ArrayList`:** Requires nesting structures (e.g., `ArrayList<ArrayList<Integer>>`), which becomes syntactically messy, inefficient, and difficult to manage. [1]

Q41. Comparable vs Comparator

The main difference is that `Comparable` provides a single natural sorting order inside the class itself, while `Comparator` defines external custom sorting logic outside the class and supports multiple sorting sequences. [1, 2]

Key Differences

Feature	Comparable	Comparator
Package	<code>java.lang</code>	<code>java.util</code>
Method	<code>compareTo(T o)</code> (takes one object)	<code>compare(T o1, T o2)</code> (takes two objects)
Sorting Type	Natural/default sorting	Custom/external sorting
Number of Orders	Only one order per class	Multiple different orders
Code Modification	⌘ You must modify the class to implement <code>Comparable<T></code> and override the <code>compareTo()</code> method.	⌘ You create a separate class or use a lambda expression to implement <code>Comparator<T></code> and override the <code>compare(T o1, T o2)</code> method.

Q42. Generics and type erasure

In Java, **Generics** and **Type Erasure** are deeply connected concepts introduced in Java 5. Their primary purpose is to provide **compile-time type safety** without introducing any runtime overhead or breaking compatibility with older Java applications. [1, 2, 3]

1. What is Type Erasure?

Type erasure is the process by which the Java compiler **removes all generic type information** during compilation. At runtime, the Java Virtual Machine (JVM) has no knowledge of the specific type parameters used in your source code. [1, 2, 3]

For example, a `List<String>` and a `List<Integer>` both compile into the exact same raw type: `List`. [1, 2]

Why did Java choose Type Erasure?

When generics were added in 2004, billions of lines of legacy Java code were already running in production. Modifying the JVM to handle distinct runtime types for generics would have broken **backward compatibility** across the ecosystem. Type erasure allowed new generic code to interoperate seamlessly with older, non-generic compiled libraries. [1, 2, 3]

Q43. PECS: ? extends vs ? super

PECS stands for Producer Extends, Consumer Super, a rule in Java generics that tells you when to use `? extends` or `? super` wildcards. [1, 2]

Producer Extends (? extends T)

- **Use when:** Your collection or data structure **produces** data that you want to **read** or get.
 - **Behavior:** It accepts `T` and any of its **subtypes** (covariant).
 - **Limitation:** You **cannot add** new items to a `? extends T` collection (except for null) because the exact subtype is unknown.
- **Example:** `List<? extends Number>` can read `Number` items safely, whether the list contains `Integer` or `Double`. [1, 2, 3]

Consumer Super (? super T)

- **Use when:** Your collection or data structure **consumes** data that you want to **write** or put.
 - **Behavior:** It accepts `T` and any of its **supertypes** (contravariant).
 - **Limitation:** When you **read** from a `? super T` collection, you can only safely treat the output as type `Object` because the exact supertype is unknown.
- **Example:** `List<? super Integer>` lets you add `Integer` objects safely into a list of `Integer` or `Number`. [1, 2, 3]

Q44. How many abstract methods can a functional interface have?

A functional interface can have exactly one abstract method. [1, 2]

However, there are a few important nuances to how this single-method rule is handled under the hood:

- **Default and Static Methods:** It can contain **any number** of default and static methods, as they already provide a concrete implementation and do not count as abstract. [1, 2]
- **Object Class Methods:** If an interface declares an abstract method that overrides one of the public methods of `java.lang.Object` (such as `equals()`, `hashCode()`, or `toString()`), that method **does not count** toward the abstract method limit.

Q45. Functional Interface vs Lambda expression

A functional interface is a type definition with a single abstract method, while a lambda expression is the inline code block that provides the implementation for that method. [1, 2]

Key Differences

- **Definition:** A [Functional Interface](#) is an interface featuring exactly one abstract method (though it can have multiple default or static methods). A **lambda expression** is an anonymous function with parameters, an arrow token (`->`), and a body (`((args) -> body)`). [1, 2]

- **Role:** The functional interface acts as the target type (data type) for the lambda expression. A lambda cannot exist in a vacuum; it requires a compatible functional interface to validate its signature at compile time. [1, 2, 3]
- **Analogy:** Think of the functional interface as the **blueprint or shape** (specifying input and output types), and the lambda expression as the **actual code/behavior** plugged into that blueprint. [1, 2]

Q46. Predicate, Supplier, Consumer, Function

In Java 8, [Predicate](#), [Consumer](#), [Supplier](#), and [Function](#) are the four core functional interfaces in the `java.util.function` package that support lambda expressions and stream operations. [1, 2]

A functional interface has only one abstract method. Each interface serves a specific purpose based on whether it accepts an input argument or returns a result. [1, 2, 3, 4]

↵ **Predicate (Predicate<T>)**

- **Input:** One argument (T)
- **Output:** Boolean (boolean)
- **Abstract Method:** `boolean test(T t)`
- **Use Case:** Conditional checks and filtering data in [Java Streams](#). [1, 2, 3, 4]

↵ **Consumer (Consumer<T>)**

- **Input:** One argument (T)
- **Output:** None (void)
- **Abstract Method:** `void accept(T t)`
- **Use Case:** Performing side-effects on an object, such as printing, logging, or saving to a database. [1, 2, 3, 4]

↵ **Supplier (Supplier<T>)**

- **Input:** None
- **Output:** Returns a result (T)
- **Abstract Method:** `T get()`
- **Use Case:** Generating or supplying values on demand, often used for lazy initialization. [1, 2, 3, 4, 5]

↵ **Function (Function<T, R>)**

- **Input:** One argument (T)
- **Output:** Returns a result (R)
- **Abstract Method:** `R apply(T t)`
- **Use Case:** Transforming or mapping data from one type/value to another. [1, 2, 3, 4]

Q47. map() vs flatMap() in Stream API

The primary difference between `map()` and `flatMap()` in the Java Stream API lies in **how they transform data and structural nesting**. While `map()` is used for **one-to-one transformations**, `flatMap()` is used for **one-to-many transformations** and **flattens nested streams**. [1, 2, 3, 4]

Direct Comparison

Feature	<code>map()</code>	<code>flatMap()</code>
Relationship	One-to-One: Processes 1 input element and outputs exactly 1 transformed element.	One-to-Many: Processes 1 input element and outputs 0, 1, or multiple elements.
Function Return Type	Returns a single value/object of type R.	Must return a Stream<R> .
Resulting Structure	Stream<R> : Keeps the original structure intact (can result in nested streams like <code>Stream<Stream<R>></code>).	Stream<R> : Merges and flattens all inner streams into a single unified stream.
Use Case	Data transformation (e.g., converting a list of Strings to uppercase or extracting an ID).	Unnesting / flattening structures (e.g., turning a <code>List<List<String>></code> into a single <code>List<String></code>).

Q48. Can we call `stream()` directly on an `int[]` array?

To create a stream from a primitive `int[]` array, use utility methods from the `Arrays` or `IntStream` classes instead: [1, 2]

- **`Arrays.stream()`**: Pass the array as a parameter to the [Arrays.stream\(\)](#) method, which returns an optimized `IntStream`:

java

```
int[] numbers = {1, 2, 3};  
IntStream stream = Arrays.stream(numbers);
```

Use code with caution.

- **`IntStream.of()`**: Pass the array directly to the `IntStream` factory method:

java

```
IntStream stream = IntStream.of(numbers);
```

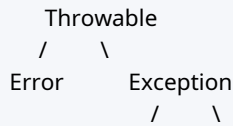
Why Avoid `Stream.of()`?

Do not use `Stream.of(numbers)` on a primitive `int[]` array. Because primitive arrays are treated as objects, `Stream.of()` will wrap the entire array as a single element, resulting in a `Stream<int[]>` rather than an `IntStream` of individual integers

Q49. Exception hierarchy

In Java, the `java.lang.Throwable` class is the root of the entire exception hierarchy. Only objects that inherit from `Throwable` can be thrown by the Java Virtual Machine (JVM) or by the `throw` statement.

The Exception Hierarchy Tree



`RuntimeException` `IOException`, `SQLException`, etc.

(Unchecked Exceptions) (Checked Exceptions)

📁 Core Components of the Hierarchy

- **Throwable:** The ultimate superclass of all errors and exceptions in the Java language.
- **Error:** Represents **serious problems** that a reasonable application should not try to catch. These are usually external to the application and caused by the environment or system resources.
 - *Examples:* `OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`.
- **Exception:** Represents **conditions that a reasonable application might want to catch**. It is divided into two main categories:

1. Checked Exceptions

These are exceptions that inherit directly from `Exception` (but not `RuntimeException`). The Java compiler **forces** you to handle or declare them at compile time.

- **Handling:** Must be enclosed in a try-catch block or declared in the method signature using the `throws` keyword.
- **Examples:** `IOException`, `FileNotFoundException`, `SQLException`, `ClassNotFoundException`.

2. Unchecked Exceptions (Runtime Exceptions)

These are exceptions that inherit from **`RuntimeException`**. The compiler does not check for these during compilation, so you are not forced to handle them explicitly. They usually indicate programming bugs.

- **Handling:** Optional, but best fixed by improving code logic (e.g., adding a null check).
- **Examples:** `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`, `IllegalArgumentException`.

Q50. Checked vs unchecked exceptions

Checked Exceptions

The compiler forces you to deal with checked exceptions. If a method can throw a checked exception, you must wrap it in a try-catch block or declare it using a `throws` keyword in the method signature. If you fail to do this, your code will not compile. [[1](#), [2](#)]

- **Examples:** `IOException`, `SQLException`, `FileNotFoundException`.
- **When to use:** For recoverable, external problems outside the program's control, such as trying to read a file that does not exist. [[1](#), [2](#), [3](#)]

Unchecked Exceptions

The compiler does not require you to catch or declare unchecked exceptions. They represent logic errors or flaws in the code that should be corrected rather than handled. [[1](#), [2](#), [3](#)]

- **Examples:** `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`.
- **When to use:** For programming bugs, invalid input arguments, or failures to meet method preconditions. [[1](#), [2](#)]

Q51. throw vs throws

The main difference is that throw is used inside a method body to actively create and trigger an exception, while throws is used in a method signature to warn callers that the method might propagate an exception. [1]

Key Differences

- **Purpose:** throw creates and launches a specific exception object. throws declares a list of potential exception types that a method might produce.
- **Location:** throw is written inside a block of code or method body. throws is written in the method or constructor signature.
- **Quantity:** throw can only launch one exception object at a time. throws can declare multiple exception classes separated by commas.
- **Handling:** throw transfers the exception immediately for handling. throws delegates the handling responsibility to the caller of the method.
- **Syntax:** throw is followed by an instance using the new keyword (e.g., throw new IOException()). throws is followed by class names (e.g., throws IOException). [1, 2, 3, 4]

Q52. Exception handling rules with method overriding

When a subclass overrides a superclass method in Java, the overriding method cannot throw any new or broader checked exceptions than the ones declared by the superclass method. [1]

Core Rules for Checked Exceptions

- **Same Exception:** The child method can throw the exact same checked exception as the parent method.
- **Subclass Exception:** The child method can throw a narrower or subclass-specific checked exception (e.g., FileNotFoundException instead of IOException).
- **No Exception:** The child method can choose to throw no exceptions at all.
- **No Broader/New Exceptions:** The child method cannot throw a broader or unrelated checked exception, nor can it add a new checked exception if the parent didn't declare one. [1, 2, 3, 4]

Rules for Unchecked Exceptions

- **Freely Thrown:** Unchecked exceptions (subclasses of RuntimeException like ArithmeticException or NullPointerException) are completely unrestricted.
- **No Constraint:** The child method can throw any unchecked exception regardless of what the parent method declares or does not declare. [1, 2]

If the Parent Method Does Not Declare Exceptions

- **No Checked Exceptions:** If the superclass method does not declare any checked exceptions, the subclass overriding method cannot declare any checked exceptions either.
- **Unchecked Allowed:** The subclass method may still throw any unchecked runtime exceptions.

Q53. What is try-with-resources?

In Java, **try-with-resources** is a specialized try statement introduced in **Java SE 7** that handles **automatic resource management**. It ensures that any opened resource—such as a file stream, database connection, or network socket—is **automatically closed** at the end of the statement execution, eliminating the need for an explicit finally block. [1, 2, 3]

To be used with this statement, a resource must be an object that implements the **java.lang.AutoCloseable** or **java.io.Closeable** interface. [1, 2]

Why Use It?

- **Prevents Resource Leaks:** It guarantees resources are closed even if the code throws an exception or exits early.
- **Cleaner Code:** It eliminates bloated boilerplate code like nesting try-catch blocks inside a finally block just to safely close resources.

- **Better Exception Handling:** If an exception is thrown both inside the try block and during the automatic closure, the original try exception is preserved, while the closing exception is safely suppressed. [\[1, 2, 3, 4, 5\]](#)

Q54. AutoCloseable vs Closeable

AutoCloseable and Closeable are Java interfaces used to release resources, but Closeable extends AutoCloseable and is specifically meant for I/O operations with stricter requirements. [\[1, 2\]](#)

Key Differences

- **Introduction:** Closeable arrived in Java 5, while AutoCloseable was added in Java 7 to support the try-with-resources statement. [\[1\]](#)
- **Exception Handling:** AutoCloseable.close() throws a broad Exception, whereas Closeable.close() throws a narrower IOException. [\[1\]](#)
- **Idempotency:** Closeable requires its close() method to be idempotent (safe to call multiple times with no extra effect); AutoCloseable does not strictly mandate idempotency, though it is recommended. [\[1, 2\]](#)
- **Use Case:** AutoCloseable fits any general resource (like database connections or custom locks), while Closeable focuses on I/O streams (like files and sockets). [\[1, 2\]](#)

Q55. Serialization vs deserialization

In Java, serialization is the process of converting an object's state into a byte stream, while deserialization is the exact reverse process—converting that byte stream back into a live Java object. [\[1, 2\]](#)

Core Differences

Feature	Serialization	Deserialization
Primary Action	Converts a Java Object → Byte Stream .	Converts a Byte Stream → Java Object .
Classes Used	ObjectOutputStream & FileOutputStream.	ObjectInputStream & FileInputStream.
Methods Used	writeObject(Object obj).	readObject().
Constructor Behavior	The constructor is called only when creating the original object.	No constructor is called. The JVM uses low-level memory allocation to recreate the object directly.
Exceptions Thrown	IOException.	IOException and ClassNotFoundException.

Q56. serialization with inheritance in java.

In Java, when a class implements [Serializable](#), all its sub-classes are serializable by default, but if a super-class is not serializable, the sub-class can still be serialized provided it implements Serializable itself. [\[1, 2\]](#)

Scenario 1: Parent Implements Serializable, Child Does Not

- **Behavior:** All child classes are serializable by default because they inherit the Serializable interface from the parent.
- **Result:** Fields from both the parent and the child classes are saved and restored successfully during serialization and deserialization. [\[1\]](#)

Scenario 2: Child Implements Serializable, Parent Does Not

- **Behavior:** The child object can be saved, but the state belonging to the non-serializable parent is ignored during serialization. [\[1, 2\]](#)
- **Result during deserialization:** Java invokes the accessible **no-arg constructor** (default constructor) of the non-serializable parent class to initialize its inherited fields with default or hardcoded constructor values. [\[1, 2, 3\]](#)
- **Requirement:** The non-serializable parent class **must** have a public or protected no-arg constructor. If it does not, a runtime exception (InvalidClassException) occurs. [\[1, 2\]](#)

Q57. Shallow copy vs deep copy

A shallow copy duplicates only the top-level properties of an object, while a deep copy recursively duplicates the entire object, including all nested objects and arrays. [\[1, 2\]](#)

Key Differences

- **Shallow Copy:** Creates a new object, but nested objects or arrays are still shared by reference with the original. Changes to nested items affect both the original and the copy. [1, 2]
- **Deep Copy:** Creates a fully independent object. Modifying nested items in the copy does not affect the original data. [1, 2]
- **Performance:** Shallow copies are faster and use less memory, whereas deep copies take more time and computer resources because they copy every nested layer. [1, 2]
 - Use a **shallow copy** when your data is flat (no nested objects) or when you want nested structures to stay linked and shared. Common tools include the [Spread Operator](#) in JavaScript or `.copy()` in Python.
- Use a **deep copy** when you need a completely separate duplicate and plan to change nested data without breaking the original source. [1, 2, 3]

Q58. Difference between wait(), sleep() and yield()

The primary difference between wait(), sleep(), and yield() is how they manage thread execution, resource ownership, and hardware scheduling. **wait() is used for inter-thread synchronization and releases its lock, sleep() pauses the current thread for a fixed duration without releasing locks, and yield() temporarily gives up CPU time to allow other equal-priority threads to run.** [1, 2, 3, 4, 5, 6]

Detailed Method Breakdown

1. wait()

- **Purpose:** Used when threads need to cooperate or communicate state changes (e.g., a producer-consumer architecture). A thread calls wait() to halt execution until another thread signals that a condition has been met. [1, 2, 3, 4]
- **Lock Behavior:** It **temporarily relinquishes the lock** on the object it is synchronizing on. This allows other threads to enter the synchronized block, execute tasks, and eventually issue a notification. [1, 2, 3, 4]

2. sleep(long millis)

- **Purpose:** Introduces a deliberate, timed delay in execution. It is commonly used for periodic tasks, polling mechanisms, or rate-limiting operations. [1, 2, 3, 4]
- **Lock Behavior:** It **retains all locks** it currently owns. If a thread sleeps inside a synchronized block, no other thread can access that synchronized resource until the sleeping thread wakes up and finishes. [1, 2, 3]

3. yield()

- **Purpose:** Provides a non-binding heuristic hint to the operating system's thread scheduler that the current thread is willing to surrender its remaining CPU time slice. [1, 2]
- **Lock Behavior:** It **does not release locks** and doesn't stop the thread. It simply places the thread back into the "Runnable" pool. If there are no other waiting threads of equal or higher priority, the scheduler may immediately return CPU execution back to the yielding thread. [1, 2, 3, 4]

Q59. Why are wait(), notify(), and notifyAll() defined in Object instead of Thread?

because they operate on intrinsic locks (monitors) that belong to every Java object, not just threads.

• **Locks Belong to Objects:** In Java, any object can act as a lock or monitor for synchronization. Because the lock state, the thread holding the lock, and the list of waiting threads are tracked per object instance, the coordination methods must be available on the object itself. [1, 2, 3]

• **Multiple Threads Can Wait on One Object:** A single shared object (resource) can have multiple threads waiting to access it. If these methods were in the Thread class, a thread would have to track and manage the status of every other thread, which is inefficient and complex. [1, 2, 3]

• **Universal Availability:** Putting them in the root Object class ensures that every object in Java inherently possesses the capability for thread coordination and inter-thread communication without requiring custom wrapper classes. [1, 2]

Q60. Difference between sleep(), wait() and join().

The primary difference is their intent: sleep() introduces a timed pause, wait() enables inter-thread communication, and join() enforces execution sequencing. [1]

Key Behavioral Differences

1. sleep(long millis)

- **How it works:** It forces the currently executing thread to temporarily halt its work for a specified duration.
- **Lock Retention:** If your thread currently holds a lock on a synchronized block, calling sleep() means it retains that lock. No other thread can access that synchronized resource until the sleeping thread wakes up and finishes its block. [1, 2, 3, 4]

2. wait()

- **How it works:** A thread calls wait() on a shared object when it cannot proceed until a certain condition is met. It enters a waiting pool.

- **Lock Release:** To prevent deadlocks, wait() immediately drops the lock on the synchronized object monitor. This grants other threads the chance to execute, change the state, and eventually call notify() or notifyAll() to wake the original thread back up. [1, 2, 3, 4, 5]

3. join()

- **How it works:** This is used when thread A depends on the final output or completion of thread B. By invoking B.join(), thread A transitions to a waiting state and will not move forward until thread B has completed its life cycle. [1, 2, 3]

// 1. Using sleep() for a simple 1-second delay

```
Thread.sleep(1000); // Current thread pauses, keeping all its locks
```

// 2. Using wait() for inter-thread communication

```
synchronized(sharedObject) {  
    while (!conditionReady) {  
        sharedObject.wait(); // Releases 'sharedObject' lock, pauses until notified  
    }  
}
```

// 3. Using join() to sequence threads

```
Thread worker = new Thread() -> {
```

```
// performing a long calculation...
```

```
});
```

```
worker.start();
```

```
worker.join(); // The main thread stops here and waits until 'worker' is totally done
```

Q61. What is ThreadLocal?

ThreadLocal is a class in Java that provides thread-local variables, meaning each thread that accesses the variable has its own independently initialized copy. It allows you to store data that is **isolated to a specific thread**, preventing data leakage or interference between different threads without using explicit synchronization (like synchronized blocks or locks).

Key Characteristics

- **Thread Isolation:** Changes made to a ThreadLocal variable by Thread A are completely invisible to Thread B.
- **No Synchronization Needed:** Because data is not shared between threads, it eliminates the performance overhead of locking and avoids race conditions.
- **Implicit Context:** It acts as a hidden parameter passed along the execution path of a single thread.

Common Use Cases

- **Database Connections:** Giving each thread its own Connection object so transactions don't conflict.
- **User Session Context:** Storing user authentication tokens or session IDs in a web application so they are accessible anywhere during that specific request's execution.
- **Thread-Unsafe Objects:** Wrapping utilities like SimpleDateFormat (which is not thread-safe) so each thread has its own dedicated, safe instance.

Q62. volatile vs Atomic classes

The primary difference is that volatile guarantees memory visibility but not atomicity, whereas **Atomic classes (like AtomicInteger)** guarantee both visibility and atomicity using lock-free, hardware-level CPU instructions. [1, 2]

Core Breakdown

Feature	volatile Keyword	Atomic Classes (java.util.concurrent.atomic)
What it is	A primitive Java keyword applied to	Wrapper classes around variables (AtomicInteger,

	variables.	AtomicLong, etc.).
Visibility	Yes. Ensures reads and writes go directly to main memory, bypassing thread-local CPU caches.	Yes. The internal values provide the same volatile-read/volatile-write visibility guarantees.
Atomicity	No. Does not prevent thread interleaving on compound operations (like count++).	Yes. Provides indivisible, thread-safe methods for compound actions (like incrementAndGet()).
Underlying Mechanism	Memory barriers / CPU cache flushing.	CAS (Compare-And-Swap) hardware-level, lock-free CPU operations.

1. Why volatile fails for compound operations

A simple operation like count++ looks like one action, but it actually consists of **three separate steps**: Read, Modify, and Write. [1, 2]

java

```
public class SharedCounter {
    // Guaranteed visibility, but NOT safe for concurrent increments
    private volatile int count = 0;

    public void increment() {
```

count++; // Race condition! Multiple threads can read the same stale value before writing.

```
}
```

If two threads call increment() at the exact same time, they might both read count = 5, both increment it to 6, and both write 6 back to main memory. One increment is completely lost. [1]

2. How Atomic Classes solve it

Atomic classes use **CAS (Compare-And-Swap)**. When a thread updates the value, the CPU atomically checks: *"Is the current value still what I think it is? If yes, update it. If no, retry."* This requires no traditional blocking locks. [1, 2, 3, 4]

java

```
import java.util.concurrent.atomic.AtomicInteger;
```

```
public class AtomicCounter {
    // Guaranteed visibility AND atomicity
    private final AtomicInteger count = new AtomicInteger(0);
```

```
    public void increment() {
```

count.incrementAndGet(); // Thread-safe! Done as a single indivisible step.

```
}
```

```
}
```

Q63. CountdownLatch vs CyclicBarrier

CountDownLatch and CyclicBarrier are both Java synchronization tools, but a [CountDownLatch](#) lets one or more threads wait for a set of operations to finish, while a CyclicBarrier makes a fixed group of threads wait for each other at a common point. [1]

Key Differences

- **Reusability:** A CountDownLatch is a one-shot tool that cannot be reset once the count hits zero. A [CyclicBarrier](#) can be reset and reused after the waiting threads are released. [1]
- **Thread Interaction:** In a CountDownLatch, worker threads call countDown() while a separate thread calls await(). In a CyclicBarrier, all participating threads call await() to wait for each other. [1, 2]
- **Actions Upon Completion:** A CyclicBarrier allows an optional Runnable task to run when the barrier trips before releasing the threads. A CountDownLatch has no built-in post-completion task. [1]

Q64. Future vs CompletableFuture

Future represents an asynchronous result that requires blocking the calling thread to retrieve data, whereas CompletableFuture extends Future to support non-blocking task chaining, combination, and callback mechanisms. [1]

Core Differences

- **Blocking vs Non-Blocking:** Future.get() blocks the thread until completion, while CompletableFuture uses callback methods like thenApply() and thenAccept() to process results asynchronously.
- **Chaining & Composition:** Future cannot combine or chain dependent tasks easily, whereas CompletableFuture allows smooth pipelines using thenCompose() or thenCombine().
- **Exception Handling:** Future lacks built-in asynchronous error handling, while CompletableFuture provides exceptionally() and handle() methods.
- **Manual Completion:** Future cannot be completed manually by your code, but CompletableFuture can be triggered programmatically via complete(). [1, 2, 3, 4, 5, 6, 7]

Q65. CompletableFuture: runAsync() vs supplyAsync()

The primary difference between CompletableFuture.runAsync() and CompletableFuture.supplyAsync() is whether the asynchronous task returns a value or not. Use supplyAsync() when you need a result from your background execution, and use runAsync() for "fire-and-forget" actions that only produce side effects. [1, 2, 3]

Direct Comparison

Feature	CompletableFuture.runAsync()	CompletableFuture.supplyAsync()
Functional Interface	Accepts a Runnable	Accepts a Supplier<T>
Input Method Lambda	() -> { ... } (No return statement)	() -> { return value; }
Return Type	CompletableFuture<Void>	CompletableFuture<T>
Primary Use Case	Logging, sending emails/notifications, writing data to a text file.	Fetching API data, querying databases, performing math calculations.

Q66. Object-level lock vs class-level lock

Object-level locking synchronizes access to a specific instance of a class, whereas class-level locking synchronizes access to the entire class across all instances. [1]

Key Differences

- **Scope:** Object-level locks tie the lock to this instance. Class-level locks tie the lock to the Class object in memory (ClassName.class).
- **Syntax:** Object-level locks use non-static synchronized methods or synchronized(this) blocks. Class-level locks use static synchronized methods or synchronized(ClassName.class) blocks.
- **Concurrency:** Multiple threads can access synchronized methods on *different* object instances simultaneously under object-level locking. Under class-level locking, no two threads can execute static synchronized methods at the same time, regardless of how many instances exist.
- **Purpose:** Object-level locks protect instance-specific data. Class-level locks protect shared static data. [1, 2, 3, 4, 5, 6]

Q67. What is ReentrantLock?

A **ReentrantLock** is an advanced synchronization utility in Java's java.util.concurrent.locks package that **provides explicit locking mechanisms** with more flexibility and features than standard synchronized blocks.

The term "reentrant" means that a thread can **acquire the same lock multiple times** without deadlocking itself, keeping track of the lock hold count.

Key Capabilities & Differences

Feature	synchronized Block	ReentrantLock
Lock Acquisition	Implicit (handled by JVM)	Explicit (via .lock())

Release Mechanism	Automatic when block exits	Manual (must use <code>.unlock()</code> in a try-finally block)
Fairness Policy	No control (threads can be starved)	Optional fairness parameter (grants lock to longest-waiting thread)
Non-blocking Attempt	Not possible	Possible via <code>.tryLock()</code> (avoids indefinite waiting)
Interruptibility	Thread cannot be interrupted while waiting	Threads can be interrupted while waiting via <code>.lockInterruptibly()</code>

Q68. How do you achieve thread safety in a multithreaded environment?

To achieve **thread safety** in a multithreaded environment, you must manage or eliminate shared mutable state. A program is thread-safe if it behaves correctly and maintains its specifications even when multiple threads execute the code concurrently without unintended interference. [\[1, 2\]](#)

The strategies for achieving thread safety fall into four main technical categories:

1. Eliminating Shared Mutable State (Thread Safety by Design)

The most effective way to handle thread safety is to avoid the need for synchronization altogether. [\[1, 2\]](#)

- **Immutability:** Make shared data unmodifiable. If an object's state cannot change after creation (e.g., using `final` or `const` qualifiers), multiple threads can safely read it simultaneously without locks. [\[1, 2, 3\]](#)
- **Statelessness:** Design components or utility functions that do not maintain any instance or global state. If a function operates solely on local variables passed into it, it is inherently thread-safe because local variables reside on the thread's private stack. [\[1, 2\]](#)
- **Thread Confinement:** Keep variables restricted to a single thread. Tools like `ThreadLocal` provide each thread with its own independent, isolated copy of a variable, entirely removing cross-thread interference. [\[1, 2\]](#)

2. Synchronization and Mutual Exclusion (Locking)

When multiple threads must modify shared data, you must coordinate access using mutual exclusion so only one thread accesses the critical region at a time. [\[1, 2, 3\]](#)

- **Implicit Locking (Pessimistic):** Utilizing built-in language primitives, such as the `synchronized` keyword in Java. It acquires an intrinsic lock on an object's monitor, forcing other threads to block and wait. [\[1, 2\]](#)
- **Explicit Locks:** Utilizing dedicated lock APIs (e.g., `ReentrantLock`, `Mutex`, `Semaphore`). These offer higher granularity and advanced controls over standard synchronization, such as timed lock acquisitions or separate read/write access permissions (`ReentrantReadWriteLock`). [\[1, 2\]](#)

3. Non-Blocking Mechanisms (Hardware-Level Solutions)

Locking can degrade performance through thread blocking and context-switching overhead. Non-blocking alternatives avoid this bottleneck: [\[1, 2\]](#)

- **Atomic Variables:** Objects like `AtomicInteger` or `AtomicReference` leverage low-level CPU instructions such as **Compare-And-Swap (CAS)**. They allow safe, lock-free updates to individual variables. [\[1, 2\]](#)
- **Volatile Variables:** In languages like Java or C++, marking a variable as `volatile` ensures memory visibility. Any thread reading the variable gets the absolute latest value flushed from main memory, rather than a stale value cached in CPU registers. *Note: Volatile only ensures visibility; it does not guarantee atomicity for compound actions like incrementing (`i++`).* [\[1, 2, 3\]](#)

4. Leveraging Concurrent Data Structures

Rather than manually wrapping collections in complex lock statements, utilize pre-built, production-tested concurrent frameworks provided by your runtime library. [\[1, 2\]](#)

- **Concurrent Collections:** Structures like `ConcurrentHashMap` or `BlockingQueue` rely on advanced partitioning and fine-grained striping locks. This allows distinct threads to modify separate parts of the data structure concurrently without blocking the entire object. [\[1, 2, 3\]](#)

Q69. Explain `ExecutorService` and types of executors

An **ExecutorService** is a framework in Java (part of the `java.util.concurrent` package) that **simplifies running tasks asynchronously** by decoupling task submission from execution details. Instead of manually creating and managing `Thread` objects, you pass your tasks (implementing `Runnable` or `Callable`) to an `ExecutorService`, which manages a pool of worker threads automatically.

Key Benefits of `ExecutorService`

- **Resource Management:** Reuses existing threads instead of creating new ones for every task, reducing memory and CPU overhead.
- **Thread Control:** Prevents system overload by capping the maximum number of concurrent threads.
- **Task Management:** Allows you to track task status, cancel tasks, and retrieve results using Future objects.

Q70. newCachedThreadPool() vs newFixedThreadPool()

newCachedThreadPool() creates new threads as needed and reuses idle ones, whereas newFixedThreadPool(n) maintains a set number of concurrent threads. [1, 2]

Key Differences

- **Core & Max Size:** newCachedThreadPool has a core size of 0 and a max size of Integer.MAX_VALUE. newFixedThreadPool keeps a fixed core and maximum size (nThreads).
- **Queue Type:** newCachedThreadPool uses a SynchronousQueue which hands off tasks directly to threads without holding them. newFixedThreadPool uses an unbounded LinkedBlockingQueue.
- **Thread Lifespan:** newCachedThreadPool kills idle threads after 60 seconds. newFixedThreadPool keeps idle threads alive forever. [1, 2, 3, 4]

Q71. Explain deadlock.

A **deadlock** is a situation in operating systems and concurrent computing where two or more processes are blocked forever because each process is holding a resource and waiting for another resource held by a different process.

The 4 Necessary Conditions for Deadlock

For a deadlock to occur, the following four conditions—known as the **Coffman conditions**—must hold true simultaneously:

- **Mutual Exclusion:** Only one process can use a resource at a time.
- **Hold and Wait:** A process is holding at least one resource and waiting to acquire additional resources held by other processes.
- **No Preemption:** Resources cannot be forcibly taken from a process; they can only be released voluntarily.
- **Circular Wait:** A closed chain of processes exists, where each process holds one or more resources that are needed by the next process in the chain.

Q72. ForkJoinPool and work stealing

In Java, **ForkJoinPool** is a specialized framework designed to optimize parallel processing on modern multi-core processors by utilizing a **divide-and-conquer strategy** and a highly efficient **work-stealing algorithm**. Introduced in Java 7, it serves as the underlying backbone for performance-critical abstractions like Java Parallel Streams and CompletableFuture. [1, 2, 3, 4]

Key Components & The Core Paradigm

The ForkJoinPool operates on the philosophy of recursive decomposition. A large problem is iteratively split (fork) into tiny subtasks until they are simple enough to solve sequentially, and then the results are merged (join) back together. [1, 2]

- **ForkJoinPool:** The central execution manager that houses a pool of worker threads, usually defaulting to the number of available CPU cores. [1, 2]
- **ForkJoinTask<V>:** The base abstraction for tasks. Rather than standard Runnable or Callable objects, you subclass:
 - **RecursiveTask<V>:** If your subtasks return a computed value.
 - **RecursiveAction:** If your tasks perform a side-effect and return void. [1, 2, 3]
- **Per-Thread Deques:** Unlike typical thread pools (like ThreadPoolExecutor) which force all threads to fight over a single shared blocking queue, **every worker thread in a ForkJoinPool maintains its own local double-ended queue (deque)**. [1, 2]

Q73. Write a program where one thread prints odd numbers and another prints even.

To have two threads print odd and even numbers sequentially without race conditions, they must synchronize and communicate. The standard approach utilizes a shared object monitor along with **wait()** and **notifyAll()** to hand off execution control dynamically. [\[1\]](#), [\[2\]](#), [\[3\]](#)

```
public class EvenOddThreadDemo {
    // Shared counter tracking the current number to print
    private int counter = 1;
    private final int limit = 10;

    // Method executed by the Odd thread
    public synchronized void printOdd() {
        while (counter <= limit) {
            // If the counter is even, this thread must wait for the even thread to finish
            while (counter % 2 == 0) {
                try {
                    wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }

            // Double-check limit condition after waking up
            if (counter <= limit) {
                System.out.println(Thread.currentThread().getName() + ": " + counter);
                counter++;
                // Notify the waiting Even thread to take over
                notifyAll();
            }
        }
    }

    // Method executed by the Even thread
    public synchronized void printEven() {
        while (counter <= limit) {
            // If the counter is odd, this thread must wait for the odd thread to finish
            while (counter % 2 != 0) {
                try {
                    wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }

            // Double-check limit condition after waking up
            if (counter <= limit) {
                System.out.println(Thread.currentThread().getName() + ": " + counter);
                counter++;
                // Notify the waiting Odd thread to take over
                notifyAll();
            }
        }
    }

    public static void main(String[] args) {
        EvenOddThreadDemo sharedPrinter = new EvenOddThreadDemo();

        // Create the odd thread using lambda expressions
```

```

Thread oddThread = new Thread() -> sharedPrinter.printOdd(), "Odd-Thread");

// Create the even thread using lambda expressions
Thread evenThread = new Thread() -> sharedPrinter.printEven(), "Even-Thread");

// Start both threads
oddThread.start();
evenThread.start();
}
}

```

Q74. Implement a bounded blocking queue using wait() and notifyAll().

Here is a thread-safe implementation of a **Bounded Blocking Queue** in Java using classic wait() and notifyAll() synchronization primitives. [\[1\]](#), [\[2\]](#)

```

import java.util.LinkedList;
import java.util.Queue;

```

```

public class BoundedBlockingQueue<T> {
    private final Queue<T> queue;
    private final int capacity;

    public BoundedBlockingQueue(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException("Capacity must be greater than 0");
        }
        this.queue = new LinkedList<>();
        this.capacity = capacity;
    }
}

```

// Adds an element to the queue. Blocks if full.

```

public synchronized void enqueue(T element) throws InterruptedException {
    // ALWAYS use a while loop to handle spurious wakeups
    while (queue.size() == capacity) {
        wait();
    }
}

```

```

queue.add(element);

```

```

// Wake up all waiting threads (producers and consumers)
notifyAll();
}

```

// Removes and returns an element from the queue. Blocks if empty.

```

public synchronized T dequeue() throws InterruptedException {
    // ALWAYS use a while loop to handle spurious wakeups
    while (queue.isEmpty()) {
        wait();
    }
}

```

```

T element = queue.poll();

```

```

// Wake up all waiting threads (producers and consumers)
notifyAll();
return element;
}

```

```
// Returns the current number of elements in the queue
public synchronized int size() {
    return queue.size();
}
}
```

Q75. Write a Java program to print a sequence up to N using 3 threads in order.

```
public class SequencePrinter {
    private final int maxLimit;
    private int counter = 1;
    private final int totalThreads = 3;
    private final Object lock = new Object();
```

```
public SequencePrinter(int maxLimit) {
    this.maxLimit = maxLimit;
}
```

```
public void printSequence(int threadRemainder) {
    while (true) {
        synchronized (lock) {
            // Wait while it's not this thread's turn OR if the counter exceeds N
            while (counter % totalThreads != threadRemainder && counter <= maxLimit) {
                try {
                    lock.wait();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }
}
```

```
// Break condition to stop threads once N is reached
if (counter > maxLimit) {
    lock.notifyAll(); // Wake up remaining threads so they can also terminate cleanly
    break;
}
```

```
// Print the current value along with the active thread name
System.out.println(Thread.currentThread().getName() + " - " + counter);
counter++;
```

```
// Notify all waiting threads to re-evaluate their condition predicates
lock.notifyAll();
}
}
}
```

```
public static void main(String[] args) {
    int N = 10; // Change this value to adjust the sequence limit
    SequencePrinter printer = new SequencePrinter(N);
```

```
// Thread 1 handles numbers where counter % 3 == 1 (1, 4, 7, 10...)
Thread t1 = new Thread(() -> printer.printSequence(1), "Thread-1");
```

```
// Thread 2 handles numbers where counter % 3 == 2 (2, 5, 8...)
Thread t2 = new Thread(() -> printer.printSequence(2), "Thread-2");
```

```
// Thread 3 handles numbers where counter % 3 == 0 (3, 6, 9...)
```

```
Thread t3 = new Thread() -> printer.printSequence(0, "Thread-3");
```

```
t1.start();  
t2.start();  
t3.start();  
}  
}
```

Q76. What is the Java Memory Model (JMM)?

The [Java Memory Model](#) (JMM) is a specification that defines how multiple threads interact through computer memory in a concurrent program, managing visibility, ordering, and atomicity. [\[1, 2, 3\]](#)

Core Concepts of JMM

- **Memory Visibility:** Determines when changes made to a shared variable by one thread become visible to other threads. Because CPUs use local caches and optimization registers, updates do not automatically show up everywhere instantly. [\[1, 2, 3\]](#)
- **Instruction Reordering:** Allows the compiler or processor to reorder code instructions for faster performance, provided the program result remains unchanged in a single-threaded context. The JMM sets rules to limit reordering when multiple threads run together. [\[1, 2\]](#)
- **Happens-Before Relationship:** A core rule that guarantees the memory effects of one specific action are visible to another action. If action A "happens-before" action B, then A's results are fully visible to B. [\[1\]](#)

How Java Enforces Rules

- **volatile keyword:** Ensures that any write to a variable goes straight to main memory, and reads pull directly from it, making updates instantly visible to other threads. [\[1, 2, 3\]](#)
- **synchronized keyword:** Restricts access to a block of code to one thread at a time and handles memory flushing upon entering and exiting the synchronized section. [\[1\]](#)
- **Memory Barriers:** Low-level hardware instructions (or fences) inserted by the JVM to stop specific types of instruction reordering and enforce data consistency. [\[1, 2, 3\]](#)

Q77. What can cause OutOfMemoryError and how do you fix it?

An OutOfMemoryError occurs when the Java Virtual Machine (JVM) cannot allocate an object because the heap is out of space and the garbage collector cannot free up any more memory. [\[1\]](#)

Causes of OutOfMemoryError

- **Memory Leaks:** Unused objects are still referenced in the code, so the garbage collector cannot delete them.
- **Small Heap Size:** The maximum memory limit set for the application is too low for the data it processes.
- **Large Data Structures:** Loading too much data into memory at once, like big lists or huge database queries.
- **Too Many Finalizers:** Finalizer threads cannot keep up with cleaning up objects before the memory fills up. [\[1, 2\]](#)

How to Fix OutOfMemoryError

- **Increase Heap Size:** Raise the maximum heap limit by adding -Xmx2048m (for 2GB) to your JVM run arguments or configuration files like gradle.properties.
- **Fix Memory Leaks:** Use a memory profiler like [Eclipse Memory Analyzer](#) or jvisualvm to find and remove unwanted object references.
- **Optimize Code:** Clear collections when done, reduce batch sizes for database queries, and avoid loading massive files all at once.
- **Analyze Dumps:** Add -XX:+HeapDumpOnOutOfMemoryError to your startup flags to create a file when the error happens, making it easier to see what filled the memory. [\[1\]](#)

Q78. What is a memory leak in Java and how do you prevent it?

A **memory leak in Java** occurs when objects that are no longer needed by the application remain referenced, preventing the Java Garbage Collector (GC) from reclaiming their memory. [\[1\]](#)

Even though Java manages memory automatically via the GC, the GC only deletes objects that are completely unreachable. If your code unintentionally retains a reference to an unused object, the GC assumes it is still needed. Over time, these forgotten objects accumulate in the heap, causing performance degradation, heavy GC overhead, and eventually crashing the application with a java.lang.OutOfMemoryError. [\[1, 2\]](#)

Common Causes & How to Prevent Them

Cause of Memory Leak	How it Happens	Prevention Strategy
Heavy Static Fields	static variables belong to the class rather than instances. They persist in memory for the entire lifetime of the application.	Minimize the use of static collections. If you must use them, explicitly clear them or set their references to null once they are no longer required.
Unclosed Resources	Database connections, file I/O streams, and network sockets open system-level handles. Leaving them open keeps their backing data structures alive in memory.	Always manage resources using try-with-resources blocks (introduced in Java 7), which automatically close resources when the block exits.
Uncleared ThreadLocal Variables	ThreadLocal data is bound to a specific thread. In web servers that reuse thread pools, this data can stick around forever if the thread never dies.	Always wrap ThreadLocal usage in a try-finally block and call the .remove() method inside the finally block to clear the thread's state.
Unregistered Listeners & Callbacks	Adding a listener or observer creates a strong reference from the event source to your object. If you discard your object but don't unregister it, the source keeps it alive.	Always unregister or deregister listeners when disposing of the object, or utilize weak references.
Unbounded Caching & Collections	Adding entries to a HashMap or ArrayList as a temporary cache without an eviction policy means the map grows indefinitely.	Use dedicated caching libraries like Caffeine or Google Guava which enforce a Time-To-Live (TTL) or size bounds. Alternatively, implement a WeakHashMap.
Non-Static Inner Classes	Non-static inner classes (and anonymous inner classes) hold an implicit hidden reference to their enclosing outer class.	Make inner classes static if they don't strictly require access to the outer class's instance variables.

Q79. Explain mark-sweep garbage collection algorithm.

The **Mark-Sweep algorithm** is a foundational, two-phase automatic memory management strategy used by runtimes like Java and JavaScript to find and reclaim memory that is no longer reachable by the program. It acts as an **indirect tracing garbage collector**, meaning instead of actively tracking down garbage, it tracks down "live" (in-use) objects and assumes anything left over is trash. [1, 2, 3, 4]

Here is how the two primary phases work:

1. The Mark Phase

Every object allocated on the heap contains a hidden metadata flag usually called the **mark bit**, which defaults to false when created. [1, 2]

- **Identify the Roots:** The algorithm begins by identifying the **GC Roots**. These are references directly accessible outside the heap, such as local variables on the call stack, CPU registers, and global/static variables. [1, 2, 3, 4]
- **Traverse the Graph:** Starting from these roots, the collector performs a tree traversal (usually a **Depth-First Search**) across all object references. [1, 2]
- **Flip the Bit:** Every time the traversal visits an object, its mark bit is switched to true. Any object that cannot be reached through this chain of references remains false. [1, 2]

2. The Sweep Phase

Once traversal is complete, the garbage collector scans the entire heap linearly from start to finish. [1, 2]

- **Reclaim Memory:** If an object's mark bit is still false, it means the program has lost all references to it. The collector unallocates this block of memory and typically adds it to a "free list" so the space can be reused for future objects. [1, 2, 3, 4]
- **Reset for Next Cycle:** If the object's mark bit is true, it survives. The collector resets its mark bit back to false so that it is ready for the next garbage collection cycle. [1, 2, 3]

Direct Comparison: Pros and Cons

Advantages	Disadvantages
Handles Cyclic References: Unlike Reference Counting, it easily handles objects that point to each other in a circle because if the whole cycle is unlinked from the roots, it won't be marked.	Memory Fragmentation: Because objects are deleted from arbitrary positions in the heap, it leaves "holes" of empty space. Over time, memory becomes fragmented, making it hard to allocate large continuous blocks.
No Runtime Overhead on Mutators: Program execution (the mutator) isn't slowed down by constantly modifying counters during normal variable assignments.	"Stop-the-World" Pauses: The execution of the main program usually must be entirely paused while the collector traverses and sweeps memory, creating noticeable lag or latency.
Simple Implementation: It relies on a straightforward, predictable execution cycle requiring minimal architectural support.	Performance Scales with Heap Size: The sweep phase must inspect <i>every</i> object in the heap, meaning execution time grows relative to total allocated memory, not just the amount of garbage.

Q80. CMS vs G1 vs ZGC.

CMS, G1, and ZGC are Java garbage collectors representing an evolution from legacy concurrent sweeping to modern ultra-low pause architectures. [\[1\]](#), [\[2\]](#)

CMS (Concurrent Mark Sweep)

- **Status:** Fully deprecated in Java 9 and completely removed in Java 14.
- **Design:** Runs major collection phases concurrently with application threads to lower pause times.
- **Drawbacks:** Does not compact memory on the fly, which causes severe memory fragmentation and occasional long "Stop-The-World" (STW) full compaction pauses. [\[1\]](#)

G1 (Garbage-First)

- **Status:** The default garbage collector for modern Java (introduced in Java 7, default since Java 9).
- **Design:** Divides the heap into equal-sized regions and prioritizes collecting regions with the most garbage first.
- **Performance:** Balances high throughput with predictable, configurable pause times (via -XX:MaxGCPauseMillis). Compacts memory incrementally to avoid fragmentation. [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#)

ZGC (Z Garbage Collector)

- **Status:** Production-ready since JDK 15, scaling from 8 GB to multi-terabyte heaps.
- **Design:** An ultra-low latency collector using colored pointers and load barriers to perform nearly all work concurrently.
- **Performance:** Delivers sub-millisecond or low-millisecond pause times that do not increase with larger heap sizes, though it trades away some raw throughput and uses higher memory overhead. [\[1\]](#), [\[2\]](#), [\[3\]](#)

Q81. GC types/use cases and Eden memory.

The Eden Space is the primary entry point for newly created objects within the Young Generation area of the Java Virtual Machine (JVM) heap. Its design relies on the **Weak Generational Hypothesis**, which states that more than 90% of Java objects are short-lived (e.g., local method variables, string concatenations) and "die young". [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#)

How Memory Flows Through Eden:

1. **Allocation:** Every time you use the new keyword, the object is instantiated directly inside the **Eden Space**. [\[1\]](#), [\[2\]](#)
 2. **Minor GC Trigger:** When the Eden space fills up, the JVM halts application threads (a brief "Stop-The-World" pause) and runs a **Minor Garbage Collection**. [\[1\]](#), [\[2\]](#)
 3. **The Survivor Ping-Pong:**
 - o **Unreferenced objects** in Eden are completely wiped out.
 - o **Live objects** are copied into the first Survivor Space (**S0**, or "From Space").
- o During the next Minor GC, surviving objects from *both* Eden and S0 are compressed and moved into **S1** ("To Space"), incrementing their **age counter**. One survivor space always remains entirely empty. [\[1\]](#), [\[2\]](#), [\[3\]](#), [\[4\]](#), [\[5\]](#)

4. **Promotion:** Once an object reaches a specific age threshold (the default tenuring threshold is **15** cycles), it is promoted to the **Old (Tenured) Generation** for long-term storage. [1]

Garbage Collector (GC) Types & Use Cases

The JVM features different garbage collection algorithms optimized for distinct core performance metrics: **Throughput** (maximal computing power), **Latency** (minimal pause times), and **Footprint** (minimal RAM overhead). [1, 2]

Garbage Collector	Primary Mechanism	Core Use Case	Best Suited For
Serial GC -XX:+UseSerialGC	Uses a single thread for all collection work; pauses all application threads during GC.	Low Footprint	Small, single-processor client machines, command-line utilities, or micro-containers (Docker) with limited RAM (< 100MB).
Parallel GC -XX:+UseParallelGC	Spawns multiple threads to clean the young/old generations simultaneously to maximize raw compute speed.	High Throughput	Batch processors, backend data miners, image rendering pipelines, and offline calculators where processing volume matters more than transient pause times.
G1 GC (Garbage-First) -XX:+UseG1GC	Divides the entire heap into small, equal logical regions (Eden, Survivor, Old). It prioritizes collecting regions filled with the most "garbage" first to hit user-defined pause targets. Default since Java 9.	Balanced Latency & Throughput	Large-scale enterprise server applications, microservices, and web servers running on multi-core processors with mid-to-large heaps (4GB - 64GB+).
ZGC / Shenandoah -XX:+UseZGC	Performs nearly all heavy compaction and marking work concurrently while application threads are actively running, limiting pauses to under 1 millisecond .	Ultra-Low Latency	High-frequency trading applications, real-time gaming backends, stream-processing nodes, and massive enterprise systems handling huge heap footprints up to several terabytes.

Q82. Java garbage collectors: when would you use which?

Choose your Java garbage collector (GC) based on your app's needs for speed, pause times, and memory size. [1, 2]

Most users on [Reddit](#) agree that matching the GC to your specific latency and throughput goals is essential for optimal performance. [1]

Serial GC

- **When to use:** Small apps with small heaps running on a single CPU.
 - **Why:** It uses one thread and has low memory overhead, but causes long pauses on big heaps. [1, 2]
- Parallel GC (Throughput Collector)**

- **When to use:** Batch processing, data pipelines, or heavy calculations where total speed matters more than short pauses.
 - **Why:** It uses multiple threads to finish work fast, but can freeze the app for longer periods. [1, 2]
- G1 GC (Garbage-First)**

- **When to use:** General web servers and large heaps (gigabytes) that need predictable pause times.
- **Why:** It splits memory into regions and lets you set a target pause time goal. It is the default setting in modern Java. [1, 2, 3]

ZGC / Shenandoah

- **When to use:** Ultra-low latency apps like financial trading platforms or live games where pauses must stay under a few milliseconds, even with massive heaps.
- **Why:** They do most cleanup work at the same time the app runs. [1, 2]

Q83. Top 2 important features of Java 8, 11, 17, 21 and 25.

Java 8

- **Lambda Expressions:** Added functional programming by letting methods act as first-class citizens and reducing boilerplate code.
- **Streams API:** Provided a declarative, pipeline-based approach to process collections of objects efficiently. [1, 2]

Java 11

- **Standard HTTP Client API:** Introduced a modern, standard [HTTP Client](#) to replace the old `HttpURLConnection` with support for HTTP/2 and WebSockets.
- **Run Single-File Source Programs:** Allowed running single .java source files directly with the java command without prior manual compilation. [1, 2, 3]

Java 17

- **Sealed Classes:** Enabled developers to restrict which classes or interfaces can extend or implement them, improving domain modeling and security.
- **Records:** Introduced a concise, compact syntax for defining plain, immutable data-carrier classes. [1, 2]

Java 21

- **Virtual Threads (Project Loom):** Brought lightweight concurrency to significantly simplify writing and scaling high-throughput applications. [1]
- **Pattern Matching for Switch:** Expanded switch expressions and statements to match patterns against data types for safer, more expressive code. [1]

Java 25

- **Scoped Values:** Finalized a flexible and safe mechanism for sharing immutable data across threads as a modern alternative to `ThreadLocal`.
- **Primitive Types in Patterns, instanceof, and switch:** Extended pattern matching capabilities to allow primitive type checks and deconstructions. [1, 2, 3]

Q84. What is a Java record? Are records immutable?

A **Java record** is a special type of class introduced in Java 14 (and finalized in Java 16) designed to act as a transparent and concise carrier for immutable data. It dramatically reduces boilerplate code by automatically generating a **canonical constructor**, public **accessor methods (getters)**, `equals()`, `hashCode()`, and `toString()` based on the defined fields. [1, 2, 3, 4]

Understanding Record Immutability

1. Shallow Immutability (Enforced by the compiler):

A record class is implicitly final (it cannot be extended), and all of its fields are implicitly final. This means that once a record instance is created, **the references held by its fields cannot be changed** to point to different objects or values. There are no setter methods. [1, 2, 3, 4, 5]

2. The Exception for Mutable Objects:

If a record contains a reference to a mutable object—such as a standard `ArrayList`, `HashSet`, or a custom mutable class—**the internal state of that object can still be modified**. [1, 2]

If you instantiate `Team` with a regular `ArrayList`, anyone with a reference to that list can add or remove elements, modifying the "immutable" record's contents. [1, 2]

How to Achieve Deep Immutability

To guarantee true, deep immutability when dealing with collections or mutable references, you must use a **compact constructor** to make defensive copies: [1, 2]

```
public record Team(String name, List<String> members) {
    // Compact constructor for defensive copying
    public Team {
        // Creates an unmodifiable copy so the internal list cannot be altered
```

```

    members = List.copyOf(members);
}
}

```

Q85. What is a sealed class?

A sealed class in Java is a class that explicitly restricts which other classes are allowed to extend it, offering an intermediate option between an open public class and a totally closed final class. [\[1, 2\]](#)

How Sealed Classes Work

- Use the sealed modifier in the class declaration.
- Use the permits clause to list the exact subclasses allowed to extend the class.
- Permitted subclasses must belong to the same package or the same named module. [\[1, 2, 3, 4\]](#)

```

java
public sealed class Shape permits Circle, Square, Rectangle {
    // class body
}

```

Use code with caution.

Rules for Permitted Subclasses

Every direct subclass of a sealed class must choose one of three modifiers to define its own inheritance status: [\[1, 2\]](#)

- **final:** Stops the inheritance chain completely for that branch.
- **sealed:** Continues the controlled hierarchy by restricting its own set of permitted subclasses.
- **non-sealed:** Re-opens the class so that any other standard class can extend it freely. [\[1, 2, 3\]](#)

Why Use Sealed Classes?

- **Controlled Inheritance:** Prevents unwanted or unauthorized extensions of a class hierarchy.
- **Exhaustive Type Handling:** Works seamlessly with modern pattern matching and switch expressions, letting the compiler verify that every possible subtype is handled.
- **Better Domain Modeling:** Accurately represents fixed algebraic data types or known domain states in code. [\[1, 2, 3, 4, 5\]](#)

Q86. What is pattern matching in modern Java?

Pattern matching in modern Java lets you test whether an object has a specific type or structure and automatically extract its data in one step. [\[1, 2\]](#)

Core Concepts

- **Target:** The object you want to test.
- **Pattern:** The combination of a type test and variable binding.
- **Pattern Variable:** A new variable automatically created and assigned if the test passes.
- **Flow Scoping:** The variable is only usable in the specific part of the code where the compiler knows the test succeeded. [\[1, 2, 3\]](#)

Key Features

- **instanceof Matching:** You no longer need to write an instanceof check followed by an explicit cast. For example, `(obj instanceof String s)` checks the type and creates a usable string variable `s` at the same time.
- **switch Matching:** You can use patterns directly inside switch expressions and statements to handle multiple types cleanly without verbosity.

- **Record Patterns:** You can destructure data carriers (records) to unpack nested values directly inside your checks. [[1](#), [2](#), [3](#), [4](#), [5](#)]

Q87. What is a virtual thread?

A **virtual thread** is a lightweight, JVM-managed thread introduced in Java 21 (via Project Loom) designed to drastically reduce the effort of writing, maintaining, and observing high-throughput concurrent applications.

Unlike traditional platform threads, which map 1:1 to operating system (OS) kernels, virtual threads are managed directly by the Java runtime and map many-to-many to OS threads.

Key Differences: Virtual Threads vs. Platform Threads

Feature	Platform Threads (Traditional)	Virtual Threads (Java 21+)
Management	Operating System (OS)	Java Virtual Machine (JVM)
Creation Cost	Expensive (High memory & CPU overhead)	Extremely cheap (Negligible overhead)
Memory Footprint	Large (~1 MB per thread stack)	Small (Starting at a few hundred bytes)
Scalability Limit	Thousands per JVM (due to OS limits)	Millions per JVM
Blocking I/O	Blocks the underlying OS thread	Unmounts from the OS thread, freeing it for other work

How They Work

Virtual threads run their code on a pool of standard OS threads (called **carrier threads**).

When a virtual thread hits a blocking operation—like waiting for a database query, reading a file, or making an API call—the JVM automatically **detaches (unmounts)** the virtual thread from its carrier thread and parks it in memory. The carrier thread is immediately free to run a different virtual thread. Once the blocking I/O operation finishes, the JVM restores the virtual thread back onto an available carrier thread to resume execution.

Main Benefit

They bring back the simple **thread-per-request** programming model without the scalability bottlenecks. Developers can write straightforward, easy-to-debug synchronous code that performs with the same high efficiency as complex, asynchronous, or reactive frameworks (like WebFlux or RxJava).

Q88. What are annotations? Can you create your own annotations?

Annotations are tags or metadata that you add to source code to provide extra information to the compiler, tools, or frameworks without changing how the code runs. [[1](#), [2](#), [3](#)]

What Are Annotations?

- **Metadata tags:** They start with an `@` symbol (such as `@Override` or `@Deprecated`).
- **Non-functional:** They do not change the direct logic or output of the code on their own.
- **Guide tools:** Compilers and frameworks read them to check rules, generate code, or manage configurations. [[1](#), [2](#), [3](#)]

Can You Create Your Own Annotations?

Yes, you can create custom annotations using the `@interface` keyword in languages like Java. [[1](#), [2](#), [3](#)]

How to Create Custom Annotations

- **Define it:** Use `public @interface AnnotationName` to declare a new custom type.
- **Add meta-annotations:** Use `@Target` to state where the tag can be placed (like methods or classes) and `@Retention` to state how long the tag stays active (like runtime or compile time).
- **Process it:** Write custom logic using reflection or compiler tools to read and act on your new tag. [1, 2, 3, 4, 5]

Q89. What built-in annotations are provided by Java?

Java provides several built-in annotations that give instructions to the compiler or provide metadata about program elements. [1, 2, 3, 4]

Common Built-in Annotations (from `java.lang`)

- **@Override:** Tells the compiler that a method is intended to override a method in a superclass. The compiler throws an error if the method does not actually override anything. [1, 2, 3]
- **@Deprecated:** Marks a class, method, or field as outdated or discouraged from use. It warns developers that the item may be removed in future versions. [1, 2]
- **@SuppressWarnings:** Instructs the compiler to ignore specific warnings (such as unused variables or unchecked casts) during compilation. [1, 2]

Meta-Annotations (from `java.lang.annotation`)

Java also includes built-in annotations used to apply metadata to *other* custom annotations: [1]

- **@Retention:** Specifies how long the annotation is kept (in source code, class files, or at runtime).
- **@Target:** Defines where an annotation can be applied (like methods, classes, or fields).
- **@Documented:** Indicates that the annotation should be included in the standard Javadoc.
- **@Inherited:** Allows an annotation to be inherited by subclasses from a superclass.
- **@Repeatable:** Allows the same annotation to be applied multiple times to a single program element. [1, 2, 3, 4]

Q90. Difference between Unit Testing and Integration Testing

Unit testing checks individual, isolated pieces of code like a single function or class, while integration testing verifies that multiple components or systems work together correctly. [1, 2]

As discussed on [StackOverflow](#), there is a clear consensus on the core distinctions between these two testing levels: [1]

Key Differences

- **Scope:**
 - *Unit:* Tests a single component, method, or function.
 - *Integration:* Examines combined modules, databases, or external APIs interacting together. [1, 2]
- **Dependencies:**
 - *Unit:* Uses fake objects, mocks, or stubs to isolate the code.
 - *Integration:* Uses real databases, servers, or closely replicated infrastructure. [1, 2, 3]
- **Execution Speed:**
 - *Unit:* Very fast, running in milliseconds.
 - *Integration:* Slower because it involves network calls, disk I/O, or external systems. [1, 2, 3]
- **Who Performs It:**
 - *Unit:* Mostly written and executed by developers.
 - *Integration:* Done by developers or quality assurance (QA) testers. [1, 2]
- **Bugs Detected:**
 - *Unit:* Logic errors in specific calculations or algorithms.
 - *Integration:* Communication, interface, or wiring issues between different layers or services. [1, 2]

Q91. How do you assert that code throws an exception in JUnit 5?

In JUnit 5, you assert that code throws an exception using the **Assertions.assertThrows()** method. This method replaces the older JUnit 4 `@Test(expected = ...)` attribute and `ExpectedException` rules. [[1](#), [2](#), [3](#), [4](#)]

The `assertThrows` method takes the **expected exception class** as its first argument and an **Executable block** (usually a lambda expression) containing the code under test as its second argument: [[1](#), [2](#)]

```
import static org.junit.jupiter.api.Assertions.assertThrows;
import org.junit.jupiter.api.Test;

public class ExceptionTest {

    @Test

    void testExceptionIsThrown() {
        // Asserts that the code block throws an IllegalArgumentException

        assertThrows(IllegalArgumentException.class, () -> {
            throw new IllegalArgumentException("Invalid input");
        });
    }
}
```

Verifying the Exception Message

`assertThrows()` **returns the thrown exception object**. You can capture this returned object to perform further assertions, such as verifying the exception message or checking custom properties: [[1](#), [2](#), [3](#)]

```
@Test

void testExceptionMessage() {
    Calculator calculator = new Calculator();

    // 1. Capture the exception instance
    ArithmeticException exception = assertThrows(ArithmeticException.class, () -> {

        calculator.divide(10, 0);
    });

    // 2. Perform assertions on the captured exception

    assertEquals("/ by zero", exception.getMessage());
}
```

Q92. @Mock vs @InjectMocks in Mockito

Key Differences

- **@Mock:**
 - Builds a fake (mock) version of a class or interface.
 - Does not run any real code inside its methods.
 - Used for dependencies that the class under test relies on (like a database or an API client). [[1](#), [2](#)]
- **@InjectMocks:**
 - Builds a real, actual instance of the class you want to test (the System Under Test).

- o Automatically finds any fields marked with `@Mock` and inserts them into this real object using constructors, setters, or field reflection.
- o Runs the actual method code of that class during the test. [[1](#), [2](#), [3](#), [4](#), [5](#)]

Q93. Mock vs Spy

A mock completely replaces a real object with a bare-bones shell, while a spy wraps an existing real instance and executes actual methods unless they are explicitly stubbed. [[1](#), [2](#)]

Key Differences

- **Mock:** Generates an empty shell where methods return default values (like null or 0) and do nothing unless told otherwise.
- **Spy:** Wraps a real object, runs real code by default, and tracks interactions.
- **Use Case for Mocks:** Best for total isolation when you want to avoid external services or slow dependencies completely.
- **Use Case for Spies:** Best for partial mocking or legacy code where most real methods should still run normally. [[1](#), [2](#), [3](#), [4](#), [5](#)]

Q94. doReturn() vs thenReturn()

In Mockito, `when(...).thenReturn(...)` calls the real method during stubbing, whereas `doReturn(...).when(...)` sets up the return value without executing the actual method. [[1](#)]

Key Differences

- **Real Method Execution:** `when(...).thenReturn(...)` invokes the target method when setting up the mock. `doReturn(...).when(...)` bypasses real method execution entirely, which prevents unwanted side effects or exceptions. [[1](#), [2](#), [3](#), [4](#)]
- **Spies:** When working with partial mocks or spies, `when(...).thenReturn(...)` executes the real method and can cause issues or failures. `doReturn(...).when(...)` safely overrides the method on a spy without calling the original implementation. [[1](#), [2](#)]
- **Type Safety:** `when(...).thenReturn(...)` provides compile-time type safety. `doReturn(...)` takes an Object parameter, meaning type mismatches are only caught at runtime via a `WrongTypeOfReturnValue` exception. [[1](#), [2](#), [3](#)]
- **Readability and Scope:** The `when(...).thenReturn(...)` syntax is generally preferred for standard mocks due to better readability and type checks. However, `doReturn(...)` is required for edge cases like stubbing void methods or re-stubbing. [[1](#), [2](#), [3](#), [4](#)]

Q95. How do you mock a void method?

You mock a void method by using special Mockito `do...()` methods like `doNothing()` or `doThrow()` instead of the standard `when().then()` syntax. [[1](#), [2](#)]

Mocking Techniques in Mockito

- **doNothing():** Instructs the void method to do nothing. This is also the default behavior for standard mocks.
 - o Syntax: `doNothing().when(mockObject).voidMethod();` [[1](#), [2](#), [3](#)]
- **doThrow():** Forces the void method to throw an exception when called.
 - o Syntax: `doThrow(new RuntimeException()).when(mockObject).voidMethod();` [[1](#), [2](#), [3](#)]
- **doAnswer():** Executes custom logic or inspects arguments passed to the void method.
 - o Syntax: `doAnswer(invocation -> { /* logic */ return null; }).when(mockObject).voidMethod();` [[1](#), [2](#), [3](#)]
- **doCallRealMethod():** Bypasses the mock and runs the actual code inside the real method.
 - o Syntax: `doCallRealMethod().when(mockObject).voidMethod();` [[1](#), [2](#)]

Q96. Constructor injection vs setter injection.

Constructor injection passes dependencies through a class constructor, while setter injection passes them through public setter methods after the object is created. [1, 2]

Constructor Injection

- **Mandatory dependencies:** Ensures an object is fully initialized and ready for use right when it is created.
- **Immutability:** Allows dependency fields to be marked as final, meaning they cannot change during the object's lifecycle.
- **Thread safety:** Safe to use in multi-threaded environments because the state cannot be modified after creation.
- **Testing:** Easy to test by passing mock dependencies directly into the constructor. [1, 2, 3]

Q97. Reactive programming: Mono and Flux with non-blocking I/O

Reactive programming is an asynchronous programming paradigm focused on data streams and the propagation of change. In the Java ecosystem, **Project Reactor** and **Spring WebFlux** use two core publisher types—**Mono** and **Flux**—to build highly scalable, **non-blocking I/O** applications. [1, 2, 3]

Instead of dedicating one thread per request (which wastes resources while waiting for database queries or API responses), reactive programming relies on an **event-loop mechanism**. Threads register a callback for I/O operations and are instantly freed up to handle other requests, maximizing hardware throughput. [1, 2, 3]

Core Core Publishers: Mono vs. Flux

Both types implement the Publisher interface from the Reactive Streams specification but serve different cardinalities: [1, 2]

Feature	Mono<T>	Flux<T>
Cardinality	Represents 0 or 1 element.	Represents 0 to N elements (potentially infinite stream).
Common Use Cases	Fetching a record by ID, saving an entity, or making a single-object REST call.	Fetching database collections, real-time data streaming (IoT, stock tickers), or video feeds.
Key Analogy	Like an HTTP response or an optional async value.	Like a live data pipeline or a continuous event listener.

Mechanics of Non-Blocking I/O

To achieve the performance benefits of reactive programming, the entire execution chain—from the HTTP request down to the database—must be non-blocking. [1, 2]

1. **The Event Loop:** A small, fixed pool of event-loop threads handles incoming connections. [1, 2]
2. **Deferred Execution (Lazy Initialization):** Creating a Mono or Flux only defines the execution pipeline. **Nothing happens until a subscription occurs** ("Nothing happens until you subscribe"). [1, 2]
3. **Delegation:** When a database query or external API call occurs, the event loop registers a callback handler with the underlying operating system container (like [Netty](#) or [R2DBC](#)) and immediately returns to accept more incoming network traffic. [1, 2]
4. **Notification:** When the data is ready, the OS notifies the runtime, and an event-loop thread picks up the data stream to feed it back into the Mono or Flux chain. [1, 2]

⚠ The Golden Rule: Never Block

Mixing blocking operations (like `Thread.sleep()`, traditional JDBC, or `RestTemplate`) inside a Project Reactor pipeline stalls the event-loop threads. Because the event-loop pool is small, blocking one thread can dramatically freeze the entire application framework. [[1](#), [2](#), [3](#)]

Non-Blocking Ecosystem Integration

A reactive application is only as fast as its slowest blocking component. To maintain end-to-end non-blocking behavior, developers swap classic blocking tools for their reactive counterparts: [[1](#)]

- **WebClient:** The non-blocking, fluent replacement for `RestTemplate` used to perform external HTTP calls.
- **R2DBC (Reactive Relational Database Connectivity):** A specification that replaces traditional JDBC to enable non-blocking communication with SQL databases like PostgreSQL or MySQL.
- **Reactive Drivers:** Standard integrations for NoSQL systems like [MongoDB](#) or [Redis](#) that natively return Mono and Flux streams. [[1](#), [2](#)]

Backpressure Support

- A fundamental advantage of Mono and Flux over standard async tools (like `CompletableFuture`) is **backpressure**. In reactive streams, the consumer controls the flow rate. If a fast database query yields millions of rows via a Flux but the calling client has a slow download speed, the subscriber can emit a demand signal (e.g., "send only 10 items at a time") to prevent the application from running out of memory. [[1](#), [2](#), [3](#), [4](#)]